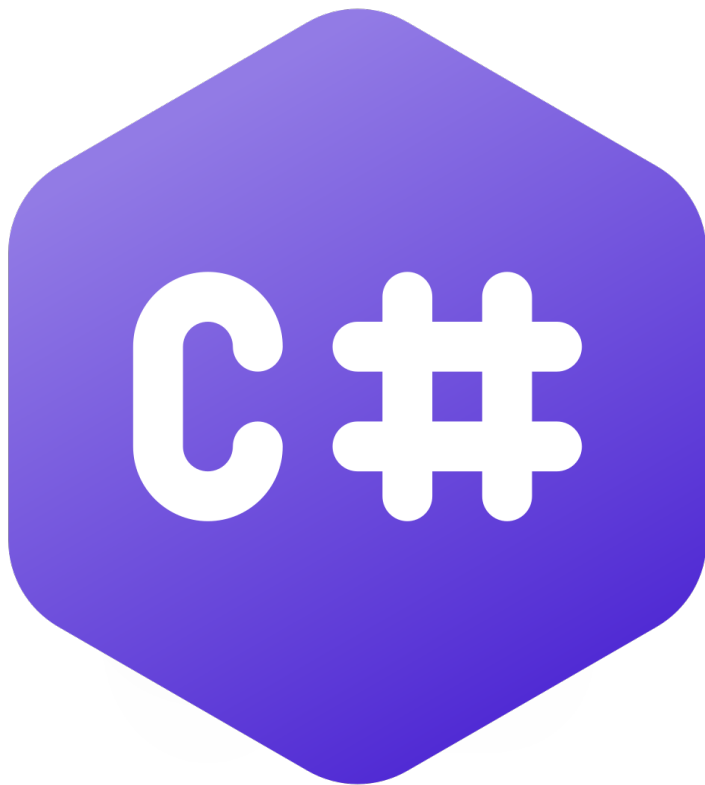


DEDICATED TO INNOVATION

www.bacancy.com





Introduction

Introduction

▶ What is C#?

- C# is pronounced "C-Sharp".
- It is an object-oriented programming language created by Microsoft that runs on the .NET Framework.
- C# has roots from the C family, and the language is close to other popular languages like C++ and Java.
- The first version was released in year 2002. The latest version, C# 13, was released in November 2024.

▶ C# is used for:

- Mobile applications, Desktop applications, Web applications, Web services, Web sites, Games, VR, Database applications, And much, much more!

Introduction

► Why Use C#?

- It is one of the most popular programming language in the world.
- It is easy to learn and simple to use.
- It has a huge community support.
- C# is an object oriented language which gives a clear structure to programs and allows code to be reused, lowering development costs.
- As C# is close to C, C++ and Java, it makes it easy for programmers to switch to C# or vice versa.

Variable and Data Type

▶ What is a Variable?

- A **variable** is like a storage box where you can keep data. Each variable has:
 - **Name:** So you can refer to it (e.g., age, name).
 - **Data Type:** So the computer knows what kind of data it will store (e.g., a number, a letter, or a true/false value).
 - **Value:** The actual data you put into the box (e.g., 25 for age).

▶ What is a Data Type?

- **Data type** tells the computer what kind of data a variable will store. In C#, you must specify this when you create a variable. It ensures the variable behaves correctly (e.g., numbers can be added, and strings can be combined).

Variable and Data Type

▶ Allowed in Variable Names

- You can start the name with a letter(a-z, A-Z).
- You can use numbers in the name, but not as the first character.
- You can use underscores in the name, and it can even be the first character.
- C# is case-sensitive, so myVariable and MyVariable are different variables.

▶ Not Allowed in Variable Names

- Variable names cannot contain spaces. Use camelCase or underscores instead.
- Variable names cannot begin with a number.
- You cannot use special characters like @, #, \$, %, ^, &, *, etc.
- C# has certain reserved keywords (e.g., int, class, namespace, etc.) that you cannot use as variable names directly.
- Hyphens are not allowed in variable names. Use underscores instead.

Data Type

C# Data Type	Range	Size	.NET Type	Notes
byte	0 to 255	1 byte	System.Byte	For small positive numbers.
sbyte	-128 to 127	1 byte	System.SByte	For small signed integers.
short	-32,768 to 32,767	2 bytes	System.Int16	For small integers.
ushort	0 to 65,535	2 bytes	System.UInt16	For small positive integers.
int	-2,147,483,648 to 2,147,483,647	4 bytes	System.Int32	Most commonly used for whole numbers.
uint	0 to 4,294,967,295	4 bytes	System.UInt32	For large positive numbers.
long	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807	8 bytes	System.Int64	For very large integers.

Data Type

C# Data Type	Range	Size	.NET Type	Notes
ulong	0 to 18,446,744,073,709,551,615	8 bytes	System.UInt64	For very large positive numbers.
float	$\pm 1.5 \times 10^{-45}$ to $\pm 3.4 \times 10^{38}$	4 bytes	System.Single	For floating-point numbers. Use f suffix (e.g., 3.14f).
double	$\pm 5.0 \times 10^{-324}$ to $\pm 1.7 \times 10^{308}$	8 bytes	System.Double	More precise than float. Default for decimal numbers.
decimal	$\pm 1.0 \times 10^{-28}$ to $\pm 7.9 \times 10^{28}$	16 bytes	System.Decimal	Best for financial or monetary calculations. Use m suffix (e.g., 12.99m).
char	Any single Unicode character	2 bytes	System.Char	Use single quotes (e.g., 'A').
bool	true or false	1 byte	System.Boolean	For logical operations.

Nullable Types

▶ Nullable Types in C#

- In C#, nullable types allow variables of value types (like int, double, bool, etc.) to hold a special value called null.

▶ Why Use Nullable Types?

- A database query may return null for an empty field.
- A variable that hasn't been assigned a value yet might need to signify "no value."

▶ How to Declare a Nullable Type

```
int? age = null; // Nullable integer
```

```
Nullable<int> age = null; // Equivalent to int?
```

readonly and const



▶ const

- A **constant** is a value that is known at **compile-time** and **never changes**.
- It must be initialized **when declared**.
- It is **implicitly static**, meaning it belongs to the type (class/struct) rather than any instance.

```
const data_type field_name = value;
```

▶ readonly

- A readonly field can only be assigned once, either:
 - At the time of declaration.
 - Or, in the constructor (for instance fields).

```
readonly data_type field_name;
```

[const](#)
[readonly](#)
[required](#)

Operators

▶ Arithmetic Operators

- Addition (+), Subtraction (-), Multiplication (*), Division (/), Modulus (%)

▶ Why Use Arithmetic Types?

- Arithmetic operators used to perform mathematical operations on numbers.

▶ Example

Addition (+), Subtraction (-), Multiplication (*), Division (/),
Modulus (%)
Examples: $a + b$, $a * b$

Operators

▶ Logical Operators

- AND (&&), OR (||), NOT (!)

▶ Why Use Logical Types?

- Logical operators are used for checking conditions

▶ Example

Examples: `a && b`, `!a`

Operators

▶ Assignment Operators

- AND (&&), OR (||), NOT (!)

▶ Why Use Assignment Types?

- Basic Assignment (=): `a = 5`
- Compound Assignments: Add and Assign (`+=`), Subtract and Assign (`-=`), Multiply and Assign (`*=`), etc.

▶ Example

Examples: `a += 5`, `b *= 2`

Arrays and Collections

➤ Array Basics

- Fixed-size collection of elements
- Elements are of the same type
- Accessed by index

➤ Lists and Dictionaries

- Lists: Dynamic-sized collection
- Dictionaries: Key-value pairs for efficient lookups

Methods and Functions

- ▶ Methods are generally the block of codes or statements in a program that gives the user the ability to reuse the same code which ultimately saves the excessive use of memory, acts as a time saver and more importantly, it provides a better readability of code. So basically, a method is a collection of statements that perform some specific task and return the result to the caller. A method can also perform some specific task without returning anything.

- ▶ **Method**

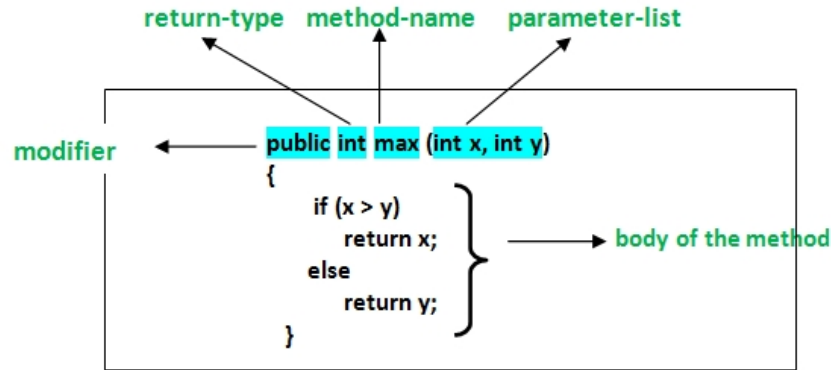
Method declaration means the way to construct method including its naming.

- ▶ **Syntax:**

`<Access_Modifier> <return_type> <method_name>([<param_list>])`

Declaration:

Methods and Functions



- ▶ In C# a method declaration consists of the following components as follows :
- ▶ **Modifier:** It defines access type of the method i.e. from where it can be accessed in your application. In C# there are Public, Protected, Private access modifiers.
- ▶ **Name of the Method:** It describes the name of the user defined method by which the user calls it or refer it. Eg. GetName()
- ▶ **Return type:** It defines the data type returned by the method. It depends upon user as it may also return void value i.e return nothing

Methods and Functions

- ▶ **Body of the Method:** It refers to the line of code of tasks to be performed by the method during its execution. It is enclosed between braces.
- ▶ **Parameter list:** Comma separated list of the input parameters are defined, preceded with their data type, within the enclosed parentheses. If there are no parameters, then empty parentheses () have to

- ▶ **Method Signature:**

Method Signature is defined by mainly two parameters(number of parameters, type of the parameters and order of the parameters), One of them is Method Name and second one is its Parameter list.

- ▶ **Method Signature:**

Method Signature is defined by mainly two parameters(number of parameters, type of the parameters and order of the parameters), One of them is Method Name and second one is its Parameter list.

- ▶ **Method Naming:**

Name of a method or a function in any programming language whether in C++ or Java or C# holds great importance and is mainly used in order to call that method for its execution. For example, findSum, computeMax, setX and getX etc.

Methods and Functions

- ▶ There are certain pre-defined rules for naming methods which a user should follow:
 - The method name must be some kind of Noun or a verb.
 - It's naming should be done in such a way that it must describe the purpose of that method.
 - The first letter of the method name can be either a small letter or a Capital letter, however, it is recommended to use the capital one.

- ▶ **Advantages of using the Methods:**

There are many advantages of using methods. Some of them are listed below:

- It makes the program well structured.
- Methods enhance the readability of the code.
- It provides an effective way for the user to reuse the existing code.
- It optimizes the execution time and memory space.

Exception Handling

- **Exception** occurs during execution of program and it breaks normal flow of program execution. If it is not handled It can cause the application to stop working.
- C# provides built-in support to handle these exceptions using try-catch blocks.
- C# has its own set of specific exceptions which can be used in an application.
- Some of them are:
 NullReferenceException,
 ArgumentOutOfRangeException,
 InvalidCastException,
 FileNotFoundException,
 InvalidOperationException etc.

Exception Handling

```
try{
```

```
    // Any suspected code that can raise exceptions should be put inside this block.
```

```
}
```

```
catch {
```

```
    // This block is an exception handler block where some action such as logging and auditing an  
    exception can be performed
```

```
}
```

```
finally {
```

```
    // his block will always be executed whether an exception raised or not. Usually, this block  
    should be used to release resources, e.g., to close any stream or file objects that were opened in  
    the try block.
```

```
}
```


Exception Handling

```
class Program {  
    static void Main(string[] args) {  
        try  
        {  
            Console.WriteLine("Enter your number: ");  
            int num = Convert.ToInt32(Console.ReadLine());  
            Console.WriteLine($"Result is ${Math.Sqrt(num)}");  
            Console.ReadKey();  
        }  
        catch (Exception ex) {  
            Console.WriteLine($"There is something wrong, please look at this message: {ex.Message}");  
            Console.ReadKey();  
        }  
    }  
}
```

//input from User is

twenty instead of 20

Self Learning

- ▶ Collections and LINQ
- ▶ Control Flow
- ▶ Enums and Structs

▶ What is a Collection?

In programming, a **collection** is a data structure used to store, manage, and manipulate multiple items (objects) as a single unit. Collections provide a way to group items, often of the same type, for operations like adding, removing, iterating, and searching.

Collections and LINQ

► Dictionary<TKey, TValue>

- A collection of key-value pairs where keys must be unique.
- **Use Case:** Efficient lookups by key.

```
var dictionary = new Dictionary<int, string>
{
    { 1, "One" },
    { 2, "Two" },
    { 3, "Three" }
};

if (dictionary.TryGetValue(2, out var value))
{
    Console.WriteLine($"Key 2: {value}"); // Output: Key 2: Two
}
```

Collections and LINQ

▶ HashSet<T>

- A collection of unique elements that does not allow duplicates.
- **Use Case:** Fast operations like checking existence, adding, or removing elements.

```
var hashSet = new HashSet<int> { 1, 2, 3, 3 }; // Duplicates are ignored

hashSet.Add(4);
hashSet.Remove(2);

Console.WriteLine(string.Join(", ", hashSet)); // Output: 1, 3, 4
```

Collections and LINQ

► LINQ Queries (Language Integrated Query)

Filtering (Where)

- **Use Case:** Filters elements based on a condition.

```
var numbers = new List<int> { 1, 2, 3, 4, 5, 6 };  
var evenNumbers = numbers.Where(n => n % 2 == 0);  
  
Console.WriteLine(string.Join(", ", evenNumbers)); // Output: 2, 4, 6
```

Ordering (OrderBy/OrderByDescending)

Use Case: Orders elements in ascending or descending order.

```
var names = new List<string> { "John", "Alice", "Bob" };  
var sortedNames = names.OrderBy(name => name);  
  
Console.WriteLine(string.Join(", ", sortedNames)); // Output: Alice, Bob, John
```

Control Flow

Conditional Statements

- If-else
- else-if
- Switch-case

Jump Statements

- Break
- Continue
- Return
- goto

Loops

- For
- ForEach
- While
- Do-While

Enums and Structs

- An enum type (or enumeration type) is a value type defined by a set of constants of with their underlying numeric type.
- To define an enumeration type, use the enum keyword and specify the names of enum members.

Enums and Structs

```
public enum Season
{
    Spring,
    Summer,
    Autumn,
    Winter
}
```

```
public class EnumConversionExample
{
    public static void Main()
    {
        Season a = Season.Autumn;
        Console.WriteLine($"Integral value of {a} is {(int)a}");

        var b = (Season)1;
        Console.WriteLine(b); // output: Summer

        var c = (Season)4;
        Console.WriteLine(c); // output: 4
    }
}
```

// output: Integral value of Autumn is 2

In this session

What we are learning in this session

- ▶ Namespaces and Assemblies
- ▶ Lambda Expressions
- ▶ Generics

Namespaces and Assemblies

▶ Namespaces

A namespace is a container for classes, interfaces, structs, enums, and delegates, used to organize code logically and avoid naming conflicts.

Key Points:

- **Syntax:** `namespace NamespaceName { // members }`
- Provides a way to group related types (e.g., `System.Collections.Generic`).
- Helps prevent **name clashes** by isolating identifiers.
- Multiple levels of namespaces can be nested.

Namespaces and Assemblies

```
namespace MyApp.Utilities
{
    public class Logger
    {
        public void Log(string message) => Console.WriteLine(message);
    }
}
```

Namespaces and Assemblies



Assemblies

An assembly is a compiled code library used for deployment, versioning, and security in .NET. It contains **Intermediate Language (IL)** code and metadata.

Types of Assemblies:

- **Executable (EXE)**: Entry point for applications.
- **Library (DLL)**: Reusable components.

Key Components:

- **Manifest**: Metadata about the assembly (name, version, culture, etc.).
- **IL Code**: Compiled .NET code.
- **Metadata**: Descriptions of types, members, and references.

Lambda Expressions

▶ Lambda Expressions

A **lambda expression** is a concise way to represent an anonymous method using the => (lambda) operator.

Syntax:

•(parameters) => expression

```
nums.Where(x => x > 5);
```

Generics

- Generic is to allow type to be a parameter to methods, classes, and interfaces.
- It enables to create classes, interfaces, and methods that work with different data types without having to define the data type explicitly.
- It provides type safety. When a generic class, interface or method is defined, the type parameter that it can work with can be specified.
- This ensures that the code will only accept and operate on data types that match the specified type parameter, preventing errors that can occur due to data type mismatches.
- C# provides a number of built-in generic types, such as `List<T>`, `Dictionary<TKey, TValue>`,

Generics

```
class DataStore<T>
{
    private T[] _data = new T[10];
    public void AddOrUpdate(int index, T item) {
        if(index >= 0 && index < 10) _data[index] =
item;
    }
    public T GetData(int index) {
        if(index >= 0 && index < 10) return
_data[index];
        else return default(T);
    }
}
```

```
DataStore<string> cities = new
DataStore<string>();
cities.AddOrUpdate(0, "Mumbai");
cities.AddOrUpdate(1, "Chicago");
cities.AddOrUpdate(2, "London");

DataStore<int> emplds = new DataStore<int>();
emplds.AddOrUpdate(0, 50);
emplds.AddOrUpdate(1, 65);
emplds.AddOrUpdate(2, 89);
```


Self Learning

Delegates and Events

- Creating and using delegates
- Anonymous methods
- Events and event handlers

Advanced C# Concepts

- Boxing and Unboxing
- Type Casting
- Extension Methods

Asynchronous Programming

- ▶ **Asynchronous Programming**
 - async and await keywords
 - Task-based asynchronous programming
 - Parallel processing with Task and Task.Run
- ▶ **Multithreading and Parallel Programming**
 - Using Thread, Task, and Parallel classes
 - Synchronization mechanisms
 - Concurrent collections
 - **Key Features:** Non-blocking I/O operations
 - Efficient resource utilization

Asynchronous Programming

- **async**: Marks a method as asynchronous, enabling the use of await.
- **await**: Suspends the method execution until the awaited task is complete.

Example

```
public async Task FetchDataAsync()
{
    var data = await GetDataFromAPI();
    Console.WriteLine(data);
}
```

Asynchronous Programming

Task Class: Represents an asynchronous operation.

Advantages

- Simplified handling of asynchronous code.
- Supports cancellation and continuations.

Example

```
Task<int> task = Task.Run(() => ComputeSum());  
int result = await task;
```

Multithreading and Parallel Programming

► Definition

Running multiple threads in parallel within the same process.

Key Objectives

- Utilize multiple CPU cores.
- Improve performance for computationally intensive operations.

Example

```
Thread t = new Thread(() =>  
    Console.WriteLine("Thread Example"));  
t.Start();
```

Multithreading and Parallel Programming

► Synchronization Mechanisms

Used to avoid data corruption in shared resources.

Mechanisms in C#

- lock: Simplest way to synchronize access.
- Monitor: Fine-grained control.
- Mutex: Inter-process synchronization.
- Semaphore: Controls access to a resource pool.

```
lock(obj)
{
    sharedResource++;
}
```

Introduction

- ▶ Object-oriented programming (OOP) is a way of developing software applications using real-world terminologies to create entities that interact with one another using objects.
- ▶ OOP makes applications flexible, reusable, well-structured, and easy to debug and test.
- ▶ The basic building blocks of OOP are classes, methods, properties, and objects.
- ▶ Classes define the structure of an object, methods represent the behavior of an object, properties hold the data of an object, and objects are instances of a class.
- ▶ **Here are some of the benefits of OOP:**
- ▶ **Reusability:** Object-oriented programming (OOP) introduces *Classes* and *Objects* to promote code reuse. Instead of repeating the same code, you can create a class once and use its object multiple times wherever needed.

Introduction

- ▶ **Extensibility:** OOP allows you to add new features without rewriting everything. Using concepts like *Inheritance* you can extend existing code easily.
- ▶ **Simplicity:** Unlike traditional modular programming, where code can be scattered and harder to secure, OOP organizes code using *Abstraction*, *Encapsulation*, and *Polymorphism*. This simplifies understanding and enhances security.
- ▶ **Maintainability:** Since OOP focuses on Reusability, Extensibility, and Simplicity, it results in clean, organized, and easy-to-maintain code. This improves the overall quality and lifespan of applications.

Classes and Objects

- ▶ **Class** : A class is a user-defined blueprint that defines both *state* (properties) and *behavior* (actions). It serves as a template for creating objects. In C#, a class consists of a name, attributes (properties), and operations (methods).
- ▶ **Object**: An object is a specific instance of a class. It brings the class to life and performs actions defined by the class. To access a class's members (attributes and methods), you use an object along with the dot(.) Operator.
- ▶ **Methods**: A method represents a particular behavior. It performs some action and might return information about an object, or update an object's data.
- ▶ **Properties**: Properties hold the data temporarily during the execution of an application.

Classes and Objects

▶ Ex:

```
public class Calculator
{
    public int CalculateSum(int no1, int no2)
    {
        return no1 + no2;
    }
}
```

```
//Creating object
Calculator calObject = new Calculator();

//Accessing Calculator class member using
Calculator class object

int result = calObject.CalculateSum(10,
20);
```

Constructors and Destructors

- ▶ **Constructor:** It is a special method present inside a class responsible for initializing the variables of that class.
- ▶ **Destructor:** A *destructor* is a special method that is called automatically when an object is destroyed or goes out of scope. It is used to release resources. In C#, a destructor has the same name as the class but is preceded by a tilde (~).

Constructors and Destructors

```
using System;
namespace ConstructorDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            ExplicitConstructor obj = new ExplicitConstructor();

            Console.ReadKey();
        }
    }
    class ExplicitConstructor
    {
        public ExplicitConstructor()
        {
            Console.WriteLine("Explicit Constructor is Called!");
        }
    }
}
```

Access Modifiers

Modifier	Accessibility
Public	Accessible from anywhere in the application.
Private	Accessible only within the class where it is defined.
Protected	Accessible within the class and by derived classes.
Internal	Accessible within the same assembly (project).
Protected Internal	Accessible within the same assembly or by derived classes.
Private Protected	Accessible within the same class and derived classes, but only in the same assembly.

Encapsulation

- ▶ **Property:** Properties hold the data temporarily during the execution of an application.
- ▶ **Fields :** Fields are variables that directly hold data in a class. Typically kept private to restrict direct access and ensure better control.
- ▶ **Accessors :** An accessor is a special block of code in a property that is used to get (retrieve) or set (assign) the value of a private field in a controlled way.

Get Accessor : The get accessor is used to retrieve the value of a private field.

Set Accessor : The set accessor is used to assign a value to a private field.

Encapsulation

```
class Product
{
    // Private field
    private string name;

    // Property with get and set accessors
    public string Name
    {
        get { return name; } // Get accessor
        set
        {
            if (!string.IsNullOrEmpty(value)) // Validation
                name = value;
            else
                Console.WriteLine("Name cannot be empty.");
        }
    }

    // Read-only property
    public string Description
    {
        get { return $"Product Name: {Name}"; }
    }

    // Constructor to initialize the field
    public Product(string productName)
    {
        Name = productName; // Uses the property
    }
}
```

```
class Program
{
    static void Main()
    {
        // Create an object
        Product product = new Product("Laptop");

        // Access and modify properties
        Console.WriteLine(product.Name); // Get accessor
        product.Name = "Smartphone"; // Set accessor
        Console.WriteLine(product.Description); // Read-only property
        //product.Description = "It is fastest smartphone"; // Error - read-only property
        // Try invalid value
        product.Name = ""; // Validation fails
    }
}
```

Encapsulation

- ▶ Encapsulation Hides the internal state and functionality of an object and only allows access through a public set of functions.
- ▶ The Encapsulation Principle in C# is very similar to a Capsule. As a capsule binds its medicine within it, in the same way in C#, the Encapsulation Principle binds the State (Variables) and Behaviour (Methods) into a single unit called class, enum, interface, etc.
- ▶ Encapsulation hides the data and implementation details show only the required members within a class, thus hiding complexity from other code. No other code needs to know about implementation detail and also can't modify the code of the class's data and methods
- ▶ Most object-oriented programming languages allow you to create classes and their properties and methods along with the access modifiers such as public, private, protected, and internal to show or hide data members and implementation details. Interfaces and abstract classes can also be used for encapsulation.

Encapsulation

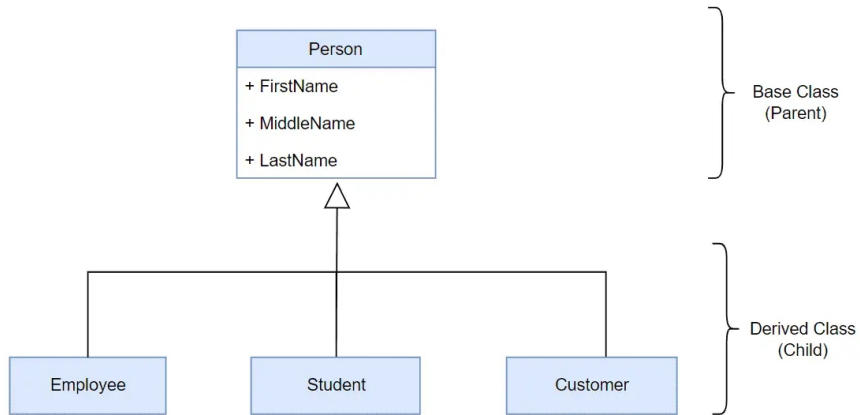
► Advantages of Encapsulation:

- Hides data and complexities.
- Restrict unauthorized access of data by allowing authorization before data access.
- Allow validation before setting data.
- Only the author of the class needs to understand the implementation, not others.
- Makes applications easy to maintain.

Inheritance

- ▶ In object-oriented programming, inheritance is another type of relationship between classes. Inheritance is a mechanism of reusing the functionalities of one class into another related class.
- ▶ Inheritance is referred to as "is a" relationship. In the real world example, a customer is a person. In the same way, a student is a person and an employee is also a person. They all have some common things, for example, they all have a first name, middle name, and last name.
- ▶ To translate this into object-oriented programming, we can create the Person class with first name, middle name, and last name properties and inherit the Customer, Student, and Employee classes from the Person class. That way we don't need to create the same properties in all classes and avoid the violation of the DRY (Do not Repeat Yourself) principle.
- ▶ Note that the inheritance can only be used with related classes where they should have some common behaviors and perfectly substitutable.

Inheritance



- ▶ In C#, use the `:` symbol to inherit a class from another class. For example, the following **Employee** class inherits from the **Person** class in C#.

▶ Example: Class Inheritance in C#

```
class Person
{
    public string FirstName { get; set; }
    public string LastName { get; set; }

    public string GetFullName(){
        return FirstName + " " + LastName;
    }
}

class Employee : Person
{
    public int EmployeeId { get; set; }
    public string CompanyName { get; set; }
}
```

Inheritance

- ▶ In the above example, the Person class is called the base class or the parent class, and the Employee class is called the derived class or the child class.
- ▶ The Employee class inherits from the Person class and so it automatically acquires all the public members of the Person class. It means even if the Employee class does not include FirstName, LastName properties and GetFullName() method, an object of the Employee class will have all the properties and methods of the Person class along with its own members.

Note: C# does not allow a class to inherit multiple classes. A class can only achieve multiple inheritance through interfaces.

Inheritance

- ▶ **Constructors:**

Creating an object of the derived class will first call the constructor of the base class and then the derived class. If there are multiple levels of inheritance then the constructor of the first base class will be called and then the second base class and so on.

- ▶ **Object Initialization:**

You can create an instance of the derived class and assign it to a variable of the base class or derived class. The instance's properties and methods are depending on the type of variable it is assigned to. Here, a type can be a class or an interface, or an abstract class.

- ▶ **The following table list supported members based on a variable type and instance type.**

Instance variable	Instance Type	Instance Members of
Base type	Base type	Base type
Base type	Derived type	Base type
Derived type	Derived type	Base and derived type

Inheritance

```
class Person
{
    public string FirstName { get; set; }
    public string LastName { get; set; }

    public string GetFullName()
    {
        return FirstName + " " + LastName;
    }
}

class Employee : Person
{
    public int EmployeeId { get; set; }
    public string CompanyName { get; set; }

    public void DisplayEmployeeDetails()
    {
        Console.WriteLine($"Employee ID: {EmployeeId}");
        Console.WriteLine($"Name: {GetFullName()}");
        Console.WriteLine($"Company: {CompanyName}");
    }
}
```

```
static void Main(string[] args)
{
    // Case 1: Base type variable, Base type instance
    Person person = new Person();
    person.FirstName = "John";
    person.LastName = "Doe";
    Console.WriteLine("Case 1: Base type variable, Base type instance");
    Console.WriteLine($"Full Name: {person.GetFullName()}");
    // person.EmployeeId = 101; // Error: Employee-specific members not accessible
    Console.WriteLine();

    // Case 2: Base type variable, Derived type instance
    Person personAsEmployee = new Employee();
    personAsEmployee.FirstName = "Jane";
    personAsEmployee.LastName = "Smith";
    Console.WriteLine("Case 2: Base type variable, Derived type instance");
    Console.WriteLine($"Full Name: {personAsEmployee.GetFullName()}");
    // personAsEmployee.EmployeeId = 102; // Error: Employee-specific members not accessible
    Console.WriteLine();

    // Case 3: Derived type variable, Derived type instance
    Employee employee = new Employee();
    employee.FirstName = "Alice";
    employee.LastName = "Brown";
    employee.EmployeeId = 202;
    employee.CompanyName = "Tech Corp";
    Console.WriteLine("Case 3: Derived type variable, Derived type instance");
    Console.WriteLine($"Full Name: {employee.GetFullName()}");
    Console.WriteLine($"Employee ID: {employee.EmployeeId}");
    Console.WriteLine($"Company Name: {employee.CompanyName}");
    employee.DisplayEmployeeDetails();
    Console.WriteLine();
}
```

Inheritance

► Important Points:

- A class can inherit a single class only. It cannot inherit from multiple classes.
- A class cannot inherit from a struct.
- A class can inherit (implement) one or more interfaces.
- A Struct can inherit from one or more interfaces. However, it cannot inherit from another struct or class.
- An interface can inherit from one or more interfaces but cannot inherit from a class or a struct.
- Constructors or destructors cannot be inherited.

Polymorphism

- ▶ Polymorphism is a Greek word that means multiple forms or shapes. You can use polymorphism if you want to have multiple forms of one or more methods of a class with the same name.
- ▶ In C#, polymorphism can be achieved in two ways:
 - Compile-time Polymorphism
 - Run-time Polymorphism
- ▶ **Compile-time Polymorphism (Method Overloading):**
 - Compile-time polymorphism is also known as method overloading. C# allows us to define more than one method with the same name but with different signatures. This is called method overloading.
 - Method overloading is also known as early binding or static binding because which method to call is decided at compile time, early than the runtime.
- ▶ **Rules for Method Overloading:**
 - Method names should be the same but method signatures must be different. Either the number of parameters, type of parameters, or order of parameters must be different.
 - The return type of the methods does not play any role in the method overloading.
 - Optional Parameters take precedence over implicit type conversion when deciding which method definition to bind.

Method Overloading

```
class MathOperations
{
    // Method with two int parameters
    public void Add(int a, int b)
    {
        Console.WriteLine("Sum of integers: " + (a + b));
    }

    // Overloaded method with three int parameters
    public void Add(int a, int b, int c)
    {
        Console.WriteLine("Sum of three integers: " + (a + b + c));
    }

    // Overloaded method with double parameters
    public void Add(double a, double b)
    {
        Console.WriteLine("Sum of doubles: " + (a + b));
    }
}
```

```
static void Main()
{
    MathOperations math = new MathOperations();

    math.Add(5, 10);           // Calls Add(int, int)
    math.Add(5, 10, 15);       // Calls Add(int, int, int)
    math.Add(5.5, 2.5);        // Calls Add(double, double)
}
```

Method Overriding

- ▶ Run-time polymorphism is also known as inheritance-based polymorphism or method overriding.
- ▶ Inheritance allows you to inherit a base class into a derived class and all the public members of the base class automatically become members of the derived class. However, you can redefine the base class's member in the derived class to provide a different implementation than the base class. This is called method overriding that also known as runtime polymorphism.
- ▶ In C#, by default, all the members of a class are sealed and cannot be redefined in the derived class. Use the virtual keyword with a member of the base class to make it overridable, and use the override keyword in the derived class to indicate that this member of the base class is being redefined in the derived class

Method Overriding

```
class Person
{
    public virtual void Greet() // Virtual method for overriding
    {
        Console.WriteLine("Hello from Person");
    }
}

class Employee : Person
{
    public override void Greet() // Overriding the base class method
    {
        Console.WriteLine("Hello from Employee");
    }
}
```

```
static void Main(string[] args)
{
    // Case 1: Base class reference, Base class object
    Person p1 = new Person();
    p1.Greet(); // Calls Person's Greet()

    // Case 2: Base class reference, Derived class object (Upcasting)
    Person p2 = new Employee();
    p2.Greet(); // Calls Employee's overridden Greet() due to polymorphism

    // Case 3: Derived class reference, Derived class object
    Employee emp = new Employee();
    emp.Greet(); // Calls Employee's Greet()
}
```

Method Overriding

- ▶ As you learned in the previous chapter the C# compiler decides which methods to call at the compile time in the compile-time polymorphism. In the run time polymorphism, it will be decided at run time depending upon the type of an object.
- ▶ To understand why method overriding is called the runtime polymorphism, look at the given example.
- ▶ In the above example, the Display() method takes a parameter of the Person type. Thus, you can pass an object of the Person type or the Employee type when you call the Display() method. The Display() method does not know the type of parameter you passed at compile time. It can be anything at runtime. That's why method overriding is called runtime polymorphism.

```
class Program
{
    // Static method that accepts a Person type but calls the actual object's method
    public static void Display(Person person)
    {
        person.Greet(); // Calls the overridden method at runtime
    }

    static void Main(string[] args)
    {
        Person p = new Person();
        Employee e = new Employee();
        Person pe = new Employee(); // Upcasting: Person reference, Employee object

        Console.WriteLine("Passing Person instance:");
        Display(p); // Calls Person.Greet()

        Console.WriteLine("\nPassing Employee instance:");
        Display(e); // Calls Employee.Greet()

        Console.WriteLine("\nPassing Employee instance as Person type:");
        Display(pe); // Calls Employee.Greet() due to runtime polymorphism
    }
}
```

Runtime Polymorphism (Method Overriding)

► Rules for Overriding:

- A method, property, indexer, or event can be overridden in the derived class.
- Static methods cannot be overridden.
- Must use virtual keyword in the base class methods to indicate that the methods can be overridden.
- Must use the override keyword in the derived class to override the base class method.

Virtual Methods

- ▶ In C#, a virtual method is a method that can be overridden in a derived class.
- ▶ When a method is declared as virtual in a base class, it allows a derived class to provide its own implementation of the method.
- ▶ To declare a method as virtual in C#, the "virtual" keyword is used in the method declaration in the base class.
- ▶ In the derived class, the method can be overridden by using the "override" keyword in the method declaration. For example.
- ▶ Using virtual methods can be useful in situations where you want to provide a base implementation in a base class, but allow derived classes to modify or extend the behavior of that method.
- ▶ We can't use the virtual modifier with the static, abstract, private, or override modifiers.

Virtual Methods

```
class Animal
{
    // Virtual method - Can be overridden in derived classes
    public virtual void MakeSound()
    {
        Console.WriteLine("Animal makes a sound");
    }

    // Non-virtual method - Cannot be overridden
    public void Sleep()
    {
        Console.WriteLine("Animal is sleeping");
    }
}

class Dog : Animal
{
    // Overriding the virtual method
    public override void MakeSound()
    {
        Console.WriteLine("Dog barks: Woof Woof!");
    }
}

class Cat : Animal
{
    // Not overriding MakeSound(), so it uses the base class version
}
```

```
static void Main()
{
    Animal myAnimal = new Animal();
    myAnimal.MakeSound(); // Calls Animal's MakeSound()
    myAnimal.Sleep();     // Calls Animal's Sleep()

    Animal myDog = new Dog();
    myDog.MakeSound();     // Calls Dog's overridden MakeSound()
    myDog.Sleep();         // Calls Animal's Sleep() (non-virtual)

    Animal myCat = new Cat();
    myCat.MakeSound();     // Calls Animal's MakeSound() (not overridden)
    myCat.Sleep();         // Calls Animal's Sleep() (non-virtual)
}
```

Abstract class and methods

- ▶ An abstract class is a class that cannot be instantiated. It is designed to be a base class and must be inherited by other classes. It can have abstract methods (without implementation) and concrete methods (with implementation).
- ▶ Abstract classes and methods are declared using the abstract keyword.
- ▶ Abstract classes cannot be instantiated directly, but they can be used as base classes for other classes that derive from them.
- ▶ A method without the body is known as the Abstract Method. What the method contains is only the declaration of the method.
- ▶ Who will Provide the Implementation of Abstract Methods? Answer is child class
- ▶ An abstract class can contain both abstract and non-abstract methods. If a child class of an abstract class wants to consume any non-abstract methods of its parent, it should implement all abstract methods.

Abstract class and methods

```
abstract class Animal
{
    public abstract void Speak(); // Abstract method (must be implemented)
}

class Dog : Animal
{
    public override void Speak()
    {
        Console.WriteLine("Dog barks: Woof Woof!");
    }
}
```

```
static void Main()
{
    // Using abstract class reference (Polymorphism)
    Animal animalRef = new Dog();
    animalRef.Speak(); // Calls Dog's Speak() method

    // Using derived class object directly
    Dog myDog = new Dog();
    myDog.Speak(); // Calls Dog's Speak() method
}
```

Interface

- ▶ The Interface in C# is a Fully Unimplemented Class used for declaring a set of operations/methods of an object.
- ▶ An interface is a contract that defines what a class should do, but not how it does it. It only contains method declarations (without implementation), and any class that implements the interface must provide implementations for all its methods.
- ▶ By default, every member of an interface is abstract, so we aren't required to use the abstract modifier on it again, just like we do in the case of an abstract class.
- ▶ We cannot declare fields/variables, constructors, and destructors in an interface in C#.
- ▶ C# does not support multiple inheritance for classes (a class cannot inherit from multiple classes) but supports multiple interface implementation. If you need a class to inherit behavior or structure from multiple sources, you can use interfaces to achieve this. This allows you to share code among different classes without the complexities of multiple-class inheritance.

Interface

```
interface IPayment
{
    void MakePayment(double amount); // Method declaration (no implementation)
}

class CreditCardPayment : IPayment
{
    public void MakePayment(double amount)
    {
        Console.WriteLine($"Payment of {amount} made using Credit Card.");
    }
}

class PayPalPayment : IPayment
{
    public void MakePayment(double amount)
    {
        Console.WriteLine($"Payment of {amount} made using PayPal.");
    }
}
```

```
static void Main()
{
    IPayment payment;

    // Using Credit Card Payment
    payment = new CreditCardPayment();
    payment.MakePayment(500.75);

    // Using PayPal Payment
    payment = new PayPalPayment();
    payment.MakePayment(300.50);
}
```

Difference between Abstract Class and Interface

Abstract Class	Interface
It contains both declaration and implementation parts.	It contains only the declaration of methods, properties, events or indexes. Since C# 8, default implementations can also be included in interfaces
Multiple inheritance is not achieved by abstract class.	Multiple inheritance is achieved by interface.
It contain constructor.	It does not contain constructor.
It can contain static members.	It does not contain static members.
It can contain different types of access modifiers like public, private, protected etc.	It only contains public access modifier because everything in the interface is public.
The performance of an abstract class is fast.	The performance of interface is slow because it requires time to search actual method in the corresponding class.

Difference between Abstract Class and Interface

Abstract Class	Interface
It is used to implement the core identity of class.	It is used to implement peripheral abilities of class.
A class can only use one abstract class.	A class can use multiple interface.
If many implementations are of the same kind and use common behavior, then it is superior to use abstract class.	If many implementations only share methods, then it is superior to use Interface.
Abstract class can contain methods, fields, constants, etc.	Interface can only contains methods, properties, indexers, events.
The keyword ":" can be used for implementing the Abstract class.	The keyword ":" and "," can be used for implementing the Interface.
It can be fully, partially or not implemented.	It should be fully implemented.

Advanced Inheritance & Polymorphism

- ▶ **New Keyword** : The new keyword is used to indicate that a method or property in a derived class is intended to hide, rather than override, a method or property in the base class.
- ▶ It's important to note that the new keyword can be used to hide any method or property, regardless of whether it's virtual or not.
- ▶ The base class method can still be accessed using a base class reference.

Usage:

- ▶ When you don't want to override but just define a new implementation of a method in the derived class.
- ▶ When both versions of the method should be accessible based on the reference type.

Advanced Inheritance & Polymorphism

```
class Parent
{
    public void Show()
    {
        Console.WriteLine("Parent class Show method");
    }
    public virtual void display()
    {
        Console.WriteLine("Parent class display method");
    }
}

class Child : Parent
{
    public new void Show() // Hides the Parent's Show() method
    {
        Console.WriteLine("Child class Show method");
    }
    public override void display() // Overrides the Parent's display() method
    {
        Console.WriteLine("Child class display method");
    }
}
```

```
static void Main(string[] args)
{
    Parent parent = new Parent();
    parent.Show();
    parent.display();

    Child child = new Child();
    child.Show();
    child.display();

    Parent parentRef = new Child();
    parentRef.Show();
    parentRef.display();
}
```

Difference between Method Overriding and Method Hiding

Method Overriding	Method Hiding (new keyword)
Runtime binding (based on object type)	Compile-time binding (based on reference type)
Derived class method is called even with base class reference (polymorphism)	Base class method is called when using base class reference
If many implementations are of the same kind and use common behavior, then it is superior to use abstract class.	If many implementations only share methods, then it is superior to use Interface.
Used when a derived class wants to modify base class behavior dynamically	Used when a derived class wants to define a completely new independent method
When the parent class reference variable points to the child class, then the method in the child class is invoked while calling.	When the parent class reference variable points to the child class, then the hidden method in the parent class is invoked while calling.

SOLID Principles

- ▶ SOLID is a set of five principles in object-oriented programming (OOP) that help developers write clean, scalable, and maintainable code.
- ▶ SOLID Design Principles represent five Design Principles used to make software designs more understandable, flexible, and maintainable. The Five SOLID Design Principles are as follows:
 1. SRP - Single Responsibility Principle
 2. OSP - Open - Closed Principle
 3. LSP - Liskov Substitution Principle
 4. ISP - Interface Segregation Principle
 5. DIP - Dependency Inversion Principle
- ▶ Applying the SOLID principles in our Application Development can make our application code more modular, easier to understand, and less error-prone when changes are made.

Single Responsibility Principle

- ▶ The SRP states that a class should have only one reason to change, meaning it should have only one responsibility. This promotes modularization and makes the code easier to understand and maintain.

```
class Employee
{
    public void CalculateSalary() { /* Salary Calculation */ }
    public void GenerateReport() { /* Report Generation */ } // ✗ Wrong: Mixing responsibilities
}
```

Problem: The class is responsible for both salary calculation and report generation, which are unrelated.

Single Responsibility Principle

- Employee handle salaries and ReportGenerator handles reports (Sepeartion of concerns).

```
class Employee
{
    public void CalculateSalary() { /* Salary Calculation */ }
}

class ReportGenerator
{
    public void GenerateReport() { /* Report Generation */ }
}
```

Open-Closed Principle

- ▶ Software entities (classes, modules, functions) should be open for extension but closed for modification.
- ▶ Once a class is written, it should be closed for modifications but open for extensions.

```
class Invoice
{
    public void ProcessInvoice(string type)
    {
        if (type == "Online") { /* Process Online Invoice */ }
        else if (type == "Offline") { /* Process Offline Invoice */ } // ✗ Wrong: Modifying code for
new types
    }
}
```

Problem: Every time a new invoice type is added, we need to modify the class, which breaks OCP.

Open-Closed Principle

- ▶ New invoice types can be added **without modifying existing code**, only by extending the base class.

```
abstract class Invoice
{
    public abstract void ProcessInvoice();
}

class OnlineInvoice : Invoice
{
    public override void ProcessInvoice() { /* Process Online Invoice */ }
}

class OfflineInvoice : Invoice
{
    public override void ProcessInvoice() { /* Process Offline Invoice */ }
}
```

Liskov Substitution Principle

- ▶ Derived classes should be replaceable for their base classes without affecting correctness.
- ▶ A subclass should be able to replace its superclass **without causing errors**.

```
public class Bird
{
    public virtual void Fly()
    {
        Console.WriteLine("This bird is flying.");
    }
}

public class Sparrow : Bird { } // ✅ Sparrow can fly, no problem

public class Penguin : Bird
{
    public override void Fly() // ❌ Penguins can't fly, but forced to implement
    {
        throw new NotImplementedException("Penguins cannot fly!");
    }
}
```

```
class Program
{
    static void Main()
    {
        Bird bird1 = new Sparrow();
        bird1.Fly(); // ✅ Works fine

        Bird bird2 = new Penguin();
        bird2.Fly(); // ❌ Throws exception at runtime!
    }
}
```

Liskov Substitution Principle

Problem :

- ▶ Penguin inherits from Bird but cannot fly, so Fly() throws an exception.
- ▶ This violates LSP because a Penguin cannot fully substitute a Bird without breaking behavior.

Solution :

- ▶ Instead of forcing all birds to have a Fly() method, we should create separate interfaces for Flying Birds and Non-Flying Birds.

Liskov Substitution Principle

```
public class Bird
{
    public void Eat()
    {
        Console.WriteLine("This bird is eating.");
    }
}

public interface IFlyable
{
    void Fly();
}

public class Sparrow : Bird, IFlyable
{
    public void Fly()
    {
        Console.WriteLine("Sparrow is flying.");
    }
}

public class Penguin : Bird
{
    public void Swim()
    {
        Console.WriteLine("Penguin is swimming.");
    }
}
```

```
class Program
{
    static void Main()
    {
        IFlyable flyingBird = new Sparrow();
        flyingBird.Fly(); // ✓ Works correctly

        Bird penguin = new Penguin();
        // penguin.Fly(); ✗ Not possible now, avoiding runtime errors!
    }
}
```


Interface Segregation Principle

- ▶ The Interface Segregation Principle states that a class should not be forced to implement interfaces it does not use.
- ▶ This principle encourages the creation of small, client-specific interfaces.

```
interface IWorker
{
    void Work();
    void Eat(); // ✗ What if robots don't eat?
}

class HumanWorker : IWorker
{
    public void Work() { /* Work */ }
    public void Eat() { /* Eat */ }
}

class RobotWorker : IWorker
{
    public void Work() { /* Work */ }
    public void Eat() { throw new Exception("Robots don't eat!"); } // ✗ Wrong
}
```

Interface Segregation Principle

Problem :

- ▶ RobotWorker is forced to implement Eat(), which does not make sense for robots.

Solution:

- ▶ HumanWorker implements both IWorkable and IEatable.
- ▶ RobotWorker only implements IWorkable, avoiding the unnecessary Eat() method.

Interface Segregation Principle

```
// Separate interface for working behavior
public interface IWorkable
{
    void Work();
}

// Separate interface for eating behavior
public interface IEatable
{
    void Eat();
}

// HumanWorker implements both interfaces because humans can work and eat
public class HumanWorker : IWorkable, IEatable
{
    public void Work()
    {
        Console.WriteLine("Human is working.");
    }

    public void Eat()
    {
        Console.WriteLine("Human is eating.");
    }
}

// RobotWorker only implements IWorkable, avoiding unnecessary Eat() method.
public class RobotWorker : IWorkable
{
    public void Work()
    {
        Console.WriteLine("Robot is working.");
    }
}
```

Dependency Inversion Principle

- ▶ Dependency Inversion Principle (DIP) states that High-level modules should not depend on low-level modules. Both should depend on abstractions.
- ▶ Abstractions should not depend on details. Details should depend on abstractions.

```
public class EmailService
{
    public void SendEmail(string message)
    {
        Console.WriteLine("Sending Email: " + message);
    }
}

public class NotificationService
{
    private EmailService _emailService = new EmailService(); // ✗ Direct dependency
    public void NotifyUser()
    {
        _emailService.SendEmail("Order Shipped!");
    }
}
```

Dependency Inversion Principle

Problem:

- ▶ NotificationService depends on EmailService directly.
- ▶ If we want to switch to SMSService, we must modify NotificationService, violating Open/Closed Principle.

Solution:

- ▶ Use Abstraction (Interfaces or Abstract Classes)
- ▶ High-level modules should depend on this abstraction instead of concrete classes.
- ▶ Apply Dependency Injection (DI)
- ▶ Decouple High-Level and Low-Level Modules

Dependency Inversion Principle

```
public interface INotification
{
    void Send(string message);
}

public class EmailService : INotification
{
    public void Send(string message)
    {
        Console.WriteLine("Sending Email: " + message);
    }
}

public class SMSService : INotification
{
    public void Send(string message)
    {
        Console.WriteLine("Sending SMS: " + message);
    }
}

public class NotificationService
{
    private readonly INotification _notification;

    public NotificationService(INotification notification) // ✅ Inject Dependency
    {
        _notification = notification;
    }

    public void NotifyUser()
    {
        _notification.Send("Order Shipped!");
    }
}
```

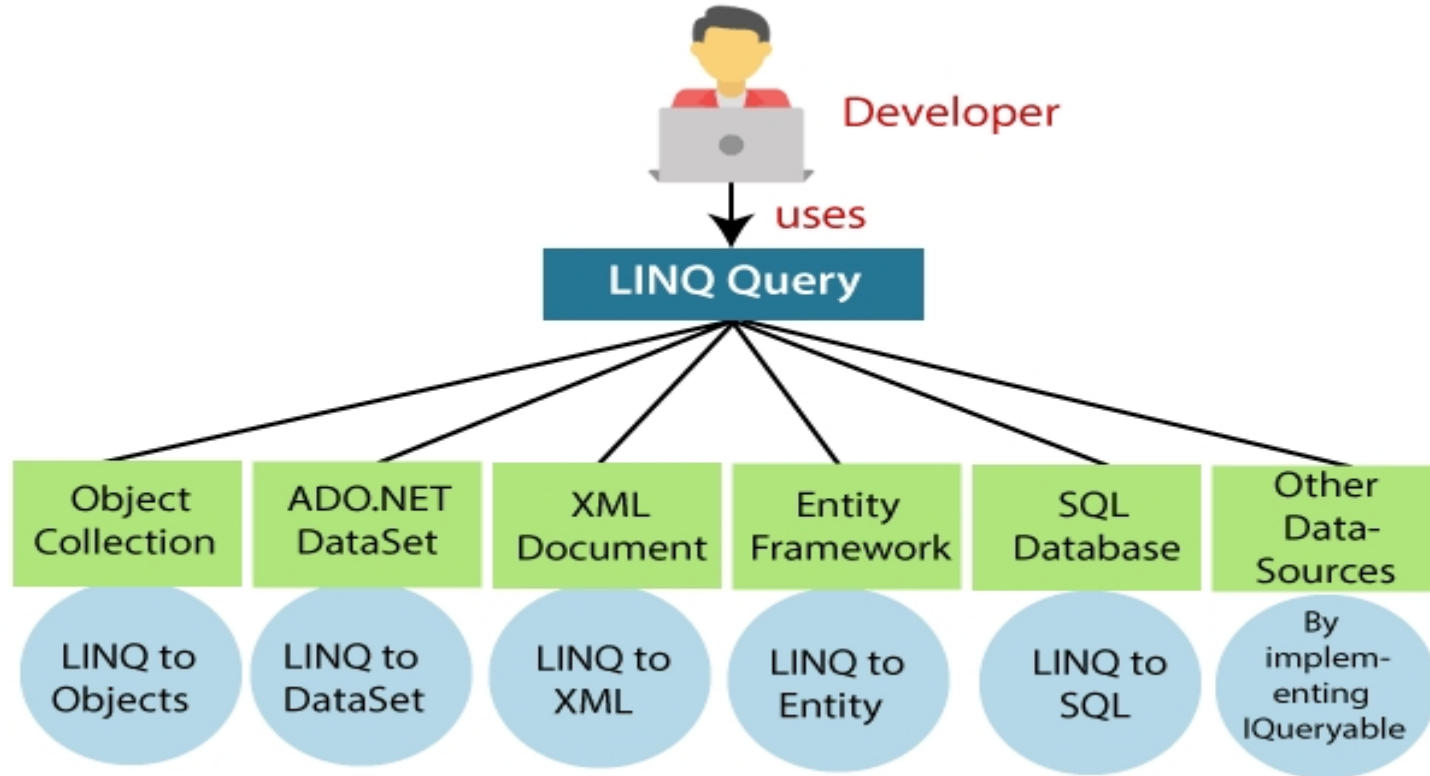
```
class Program
{
    static void Main()
    {
        INotification notification = new EmailService(); // Easily switch to SMSService
        NotificationService notificationService = new NotificationService(notification);
        notificationService.NotifyUser();
    }
}
```

Linq - Introduction

- ▶ **LINQ**, which stands for **Language Integrated Query**, is a feature in programming languages like C# and VB.NET that allows developers to write queries directly within the programming language to interact with data.
- ▶ LINQ provides a consistent model for querying different types of data sources, such as
databases, collections, XML, and more.
- ▶ Example : Query a list of numbers in C#

```
int[] numbers = { 1, 2, 3, 4, 5 };  
var evenNumbers = numbers.Where(n => n % 2 == 0);
```

Linq - Usage



Linq - Query Syntax

- ▶ Query syntax is **similar to SQL**.
- ▶ Starts with **from** clause and can be end with **Select** or **Group By** clause.

```
//Data Source
List<int> integerList = new List<int>()
{
    1, 2, 3, 4, 5, 6, 7, 8, 9, 10
};
//LINQ Query using Query Syntax
var QuerySyntax = from obj in integerList
                  where obj > 5
                  select obj;
//Execution
foreach (var item in QuerySyntax)
{
    Console.WriteLine(item + " ");
}
```

Initialization

Condition

Selection

Linq - Method Syntax

- ▶ Method syntax like *calling* an *extension* method already included.
- ▶ It uses a lambda expression to define the condition for the Query and implicitly called using variable - var to hold result of the query.

```
// String Collection
IList<string> stringList = new List<string>() {
    "May",
    "June",
    "July",
    "August"
};

// Method Query
var result = stringList.Where(s => s.Contains("May"));
```

Lambda Expression



Extension Method

Linq - Filtering and Sorting

- ▶ **Filtering:** *Where()* method filters elements.

```
var filteredNumbers = numbers.Where(n => n > 2);
```

- ▶ **Sorting:** data with *OrderBy*, *OrderByDescending*, *ThenBy*, *ThenByDescending*

```
var sortedNumbers = numbers.OrderBy(n => n);  
var descSortedNumbers = numbers.OrderByDescending(n => n);
```

- ▶ Working with **nullable values** in queries

```
List<int?> nullableNumbers = new List<int?>  
{1, null, 3, 5, null};  
var nonNullNumbers = nullableNumbers.Where(n => n.HasValue)  
.Select(n => n.Value);
```

Projection in Linq

- ▶ **Select:** Selecting specific properties using *Select()*:

```
IEnumerable<string> list = stringList.Select(s => s);  
IEnumerable<int> lengthList = stringList.Select(s => s.Length);
```

- ▶ **Select Many:** Flattening Collection using *SelectMany()*

- Applied on nested collections

```
IEnumerable<char> characterList = stringList.SelectMany(s => s);
```

- ▶ **Creating Anonymous Types:**

```
var lengthProjection = stringList.  
    Select(s => new { Month = s, Length = s.Length });
```

Linq - Aggregations Functions

- ▶ Using aggregation methods like *Count*, *Sum*, *Average*, *Min*, *Max*

```
int count = numbers.Count();  
int sum = numbers.Sum();
```

- ▶ Grouping data with *GroupBy*

```
var groupedEmployees = employees.GroupBy(e => e.Department);
```

- ▶ Selecting aggregated results in projections

```
var departmentStats = employees.GroupBy(e => e.Department)  
    .Select(g => new  
    {  
        Department = g.Key,  
        EmployeeCount = g.Count(),  
        AverageSalary = g.Average(e => e.Salary)  
    });
```

Linq - Grouping Queries

- ▶ Grouping data using *GroupBy* and *ToLookup*

```
var groupedEmployees = employees.GroupBy(e => e.Department);  
var lookupEmployees = employees.ToLookup(e => e.Department);
```

- ▶ Aggregating grouped data with custom selectors

```
var departmentStats = employees.GroupBy(e => e.Department)  
    .Select(g => new  
    {  
        Department = g.Key,  
        EmployeeCount = g.Count(),  
        MaxSalary = g.Max(e => e.Salary)  
    });
```

- ▶ Creating Nested Queries and Subqueries in LINQ

```
var departmentsWithManyEmployees = employees.GroupBy(e => e.Department)  
    .Where(g => g.Count() > 3)  
    .Select(g => new  
    {  
        Department = g.Key,  
        Employees = g.Select(e => e.Name) // Subquery to get employee names  
    });
```

Linq - Set Operations

- ▶ **Distinct:** Removes duplicates from collections:

```
List<int> integerListFirst = new List<int>{ 1, 2, 3, 4, 5, 4 };  
List<int> integerListSecond = new List<int> { 4, 5, 6, 7, 8 };  
var distinct = integerListFirst.Distinct();
```

- ▶ **Union:** Combines two sequences, removing duplicates:

```
var union = integerListFirst.Union(integerListSecond);
```

- ▶ **Intersect:** Finds Common elements between two sequences:

```
var intersect = integerListFirst.Intersect(integerListSecond);
```

- ▶ **Except:** Returns elements from the first sequence not found in the second:

```
var except = integerListFirst.Except(integerListSecond);
```

Linq - Deferred vs Immediate Execution

- ▶ **Deferred Execution:** LINQ queries are not executed when they are defined. Instead, they are executed when the result is enumerated.

```
var numbers = new List<int> { 1, 2, 3, 4, 5 };

// Deferred execution
var query = numbers.Where(n => n > 2);
// Data changes
numbers.Add(6);

// Query is executed here
foreach (var num in query){
    Console.WriteLine(num);
}
```

- ▶ Queries are re-evaluated each time they are enumerated.
- ▶ Useful for working with large datasets or when the data might change before execution.

Linq - Deferred vs Immediate Execution

- ▶ **Immediate Execution:** Forces the query to execute immediately and return the result as a collection (e.g., ToList, ToArray, ToDictionary)

```
// Immediate execution using ToList
var result = numbers.Where(n => n > 2).ToList();

// Data changes
numbers.Add(6);

// Query is already executed, so changes are not reflected
foreach (var num in result)
{
    Console.WriteLine(num);
}
```

- ▶ Give results immediately and don't want the query to be re-evaluated.
- ▶ Useful for caching or small datasets.

Linq - Resources

Resource Link:

- ▶ <https://learn.microsoft.com/en-us/dotnet/csharp/linq/>
- ▶ <https://dotnettutorials.net/lesson/introduction-to-linq/>
- ▶ <https://www.javatpoint.com/linq>

.Net core - Introduction

- ▶ **What is the ASP.NET Core?**
- ▶ ASP.NET Core is not an upgraded version of ASP.NET.
- ▶ ASP.NET Core is completely rewriting that work with the .net Core framework. It is much faster, configurable, modular, scalable, extensible, and has cross-platform support.
- ▶ It can work with both .NET Core and .net framework via the .NET standard framework. It is best suitable for developing cloud-based such as web applications, mobile applications, and IoT applications.
- ▶ ASP.NET Core was primarily designed to make the most important part of the ASP.NET components under the concept learn and the compose framework where the previous ASP.NET components were released under a variety of different licenses periodically, The ASP.NET Core framework is a completely open-source framework.
- ▶ Apart from the other parts of the framework of the .NET framework libraries, the ASP.NET Core is primarily designed from scratch to be the platform-agnostic that performs seamlessly. It will allow the ASP.NET Core apps to be deployed on the various platforms or the o/s such as the macOS or Linux-based servers or certain devices.

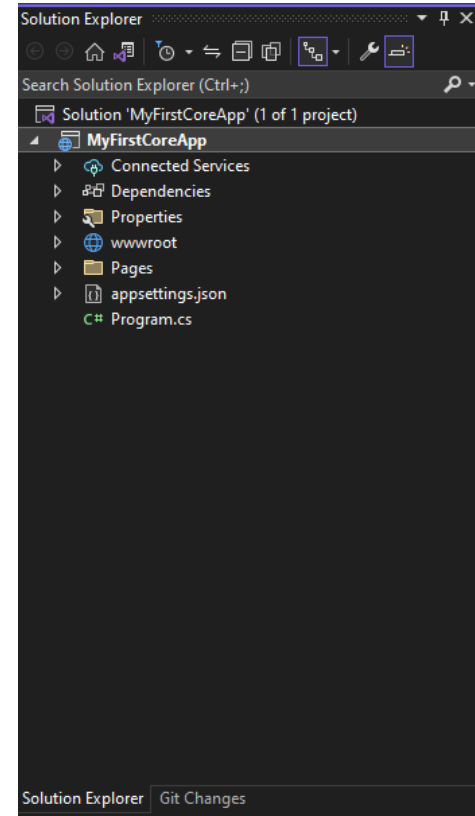
.Net core - Introduction

► What are the differences between .NET Core and .NET Framework?

.NET Core	.NET Framework
Completely open-source.	Few components are open-source.
Compatible with Linux, Windows, and Mac operating systems.	Compatible with only Windows.
Does not support desktop application development.	Supports web and desktop application development.
Supports microservices development.	Supports web and desktop application development.
Lightweight for Command Line Interface (CLI).	Heavy for Command Line Interface.
Is flexible in installation, i.e. you can install components on a needed basis.	Must be installed as a whole package

.Net core - Project Structure

- ▶ Here, we will learn about the project structure and significance of each file created by ASP.NET Core 6.0 application template using Visual Studio 2022. The following is a default project structure when you create an empty ASP.NET Core application in Visual Studio.
- ▶ **Note:** ASP.NET Core project files and folders are synchronized with physical files and folders. If you add a new file or folder in project folder then it will directly reflect in the solution explorer. You don't need to add it in the project explicitly by right clicking on the project.
- ▶ **.csproj**
ASP.NET Core 6.0 no longer uses .xproj and project.json files. Instead, it fully utilizes the .csproj file to manage the project.
- ▶ With the new version, you can edit the .csproj settings directly in Visual Studio or any text editor. To do so, right-click on the project in Visual Studio and select "Edit Project File" to modify the .csproj content.



.Net core - Project Structure

- ▶ This change was introduced in ASP.NET Core 2.0, and it carries forward into ASP.NET Core 6.0. The .csproj file now contains all the necessary project configuration and references, making it more streamlined and familiar for developers working with .NET projects.

```
<Project Sdk="Microsoft.NET.Sdk.Web">

  <PropertyGroup>
    <TargetFramework>net6.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>

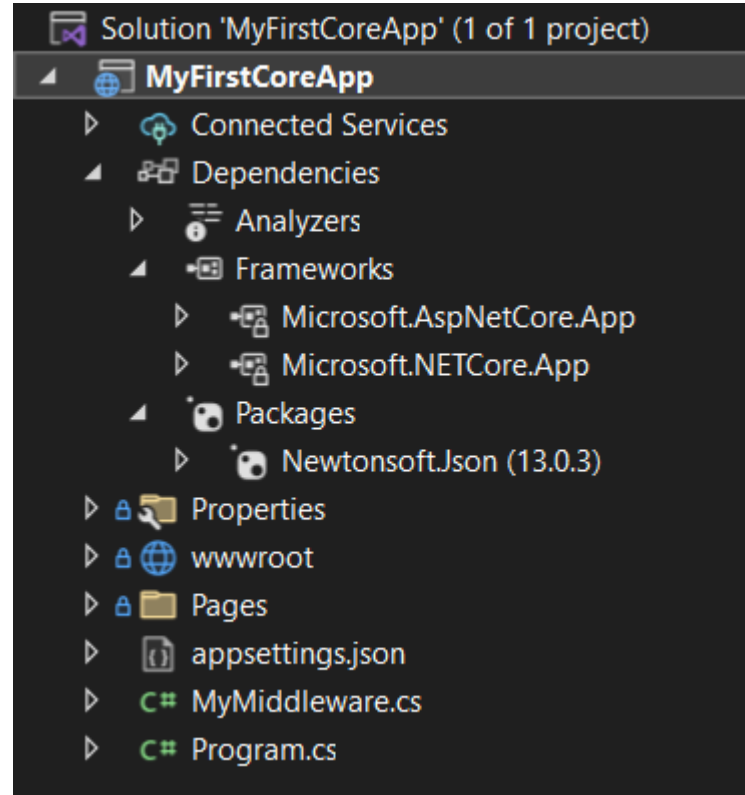
  <ItemGroup>
    <PackageReference Include="Newtonsoft.Json" Version="13.0.3" />
  </ItemGroup>

</Project>
```

- ▶ The .csproj for the above project looks like below.
- ▶ The .csproj file includes settings related to targeted .NET Frameworks, project folders, NuGet package references etc.

NuGet Package Manager

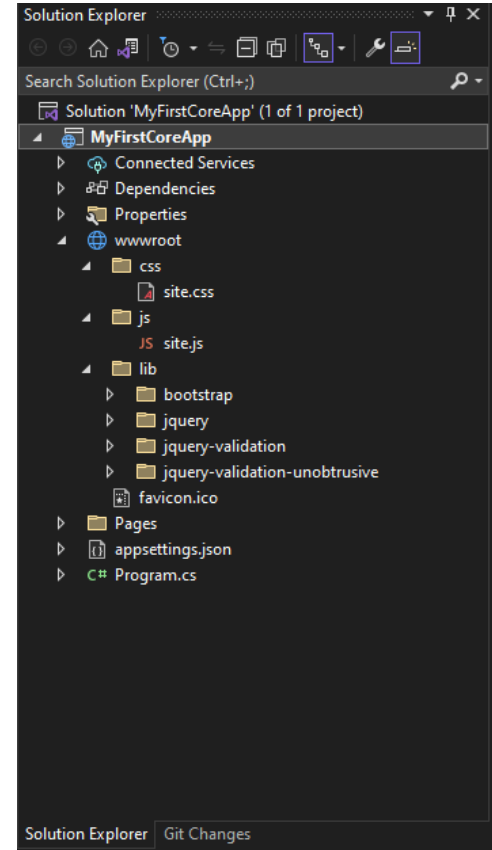
- ▶ **Dependencies:** The Dependencies in the ASP.NET Core 6.0 project contain all the installed server-side NuGet packages, as shown below.
- ▶ Right click on "Dependencies" and then click "Manage NuGet Packages.." to see the installed NuGet packages
- ▶ You can install all other required server side dependencies as NuGet packages from Manage NuGet Packages window or using Package Manager Console.



.Net core - Project Structure

▶ ASP.NET Core - wwwroot Folder

- ▶ By default, the wwwroot folder in the ASP.NET Core project is treated as a web root folder. Static files can be stored in any folder under the web root and accessed with a relative path to that root.
- ▶ In the standard ASP.NET application, static files can be served from the root folder of an application or any other folder under it. This has been changed in ASP.NET Core. Now, only those files that are in the web root - wwwroot folder can be served over an http request. All other files are blocked and cannot be served by default.
- ▶ Generally, there should be separate folders for the different types of static files such as JavaScript, CSS, Images, library scripts etc. in the wwwroot folder as shown in the given screenshot.



Routing in ASP.NET Core

- ▶ **What is Routing?**
 - ▶ Mechanism to map incoming HTTP requests to the appropriate controller actions.
- ▶ **Types of Routing:**
 - ▶ Conventional Routing
 - ▶ Attribute Routing

Routing in ASP.NET Core

- ▶ **Conventional Routing:**

- ▶ Defined in Startup.cs using `app.UseEndpoints`.
- ▶ Follows a predefined pattern (e.g., `{controller=Home}/{action=Index}/{id?}`).

- ▶ **Attribute Routing:**

- ▶ Defined directly above controllers or actions using `[Route]` attribute.
- ▶ More flexible and explicit.

Routing in ASP.NET Core

▶ Conventional Routing:

```
app.UseEndpoints(endpoints =>
{
    endpoints.MapControllerRoute(
        name: "default",
        pattern: "{controller=Home}/{action=Index}/{id?}");
});
```

▶ Attribute Routing:

```
[Route("api/[controller]")]
public class OrdersController : ControllerBase
{
    [HttpGet("{id:int}")]
    public IActionResult GetOrder(int id) { ... }
}
```

Common Status Code And HTTP Verbs

▶ Common Status Codes:

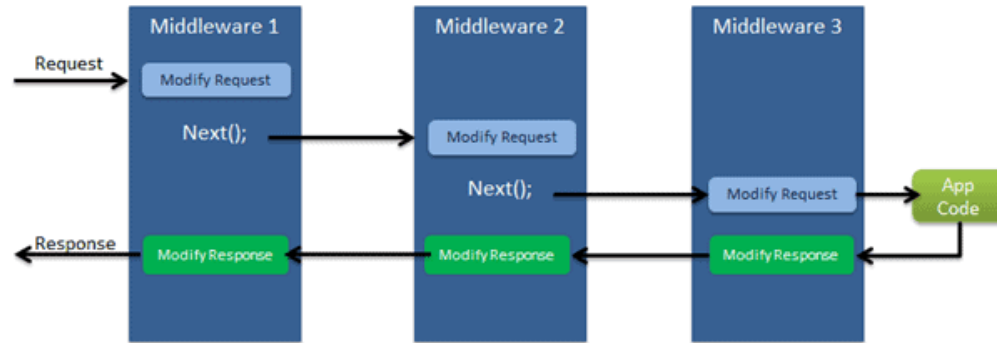
- ▶ 200 OK - Success
- ▶ 400 Bad Request - Invalid input
- ▶ 404 Not Found - Resource not found
- ▶ 500 Internal Server Error - Server error

▶ Common HTTP Verbs:

- ▶ GET - Retrieve data
- ▶ POST - Create data
- ▶ PUT - Update data
- ▶ DELETE - Delete data

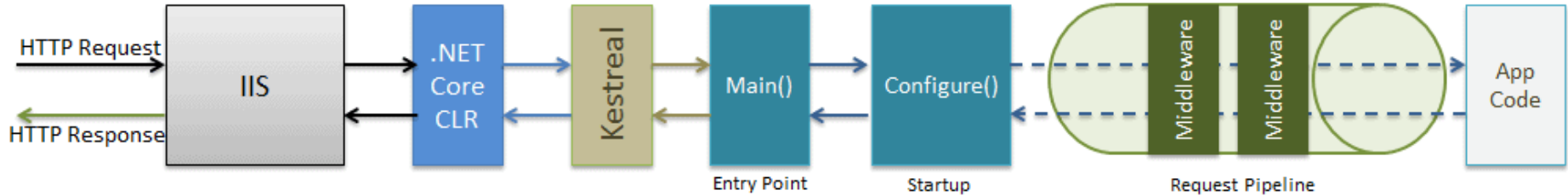
.Net core - Middleware

- ▶ **ASP.NET Core - Middleware:** ASP.NET Core introduced a new concept called Middleware. A middleware is nothing but a component (class) which is executed on every request in ASP.NET Core application. In the classic ASP.NET, HttpHandlers and HttpModules were part of request pipeline. Middleware is similar to HttpHandlers and HttpModules where both needs to be configured and executed in each request.



- ▶ Typically, there will be multiple middleware in ASP.NET Core web application. It can be either framework provided middleware, added via NuGet or your own custom middleware. We can set the order of middleware execution in the request pipeline. Each middleware adds or modifies http request and optionally passes control to the next middleware component. The following figure illustrates the execution of middleware components.
- ▶ Middlewares build the request pipeline. The following figure illustrates the ASP.NET Core request processing.

.Net core - Middleware



- ▶ **Configure Middleware:** We can configure middleware in the Program.cs file using WebApplication instance. The following example adds a single middleware using Run method which returns a string "Hello world from 1st middleware" on each request.
- ▶ In the given example, Run() is an extension method on WebApplication instance which adds a terminal middleware to the application's request pipeline. The above configured middleware returns a response with a string "Hello world from 1st middleware" for each request.

```
app.Run(async (context) => {  
    await context.Response.WriteAsync("Hello world from 1st middleware");  
});
```

.Net core - Middleware

- ▶ Mostly there will be multiple middleware components in ASP.NET Core application which will be executed sequentially. The Run method adds a terminal middleware so it cannot call next middleware as it would be the last middleware in a sequence. The following will always execute the first Run method and will never reach the second Run method.
- ▶ To configure multiple middleware, use Use() extension method. It is similar to Run() method except that it includes next parameter to invoke next middleware in the sequence.

```
app.Run(async (context) => {  
    await context.Response.WriteAsync("Hello world from 1st middleware");  
});  
  
app.Run(async (context) => {  
    await context.Response.WriteAsync("Hello world from 2nd middleware");  
});
```

- ▶ **Add Built-in Middleware Via NuGet:** ASP.NET Core is a modular framework. We can add server side features we need in our application by installing different plug-ins via NuGet. There are many middleware plug-ins available which can be used in our application.

.Net core - Dependency

Injection

- ▶ **ASP.NET Core - Dependency Injection:** ASP.NET Core is designed from scratch to support Dependency Injection. ASP.NET Core injects objects of dependency classes through constructor or method by using built-in IoC container.
- ▶ **Built-in IoC Container**
- ▶ ASP.NET Core framework contains simple out-of-the-box IoC container which does not have as many features as other third party IoC containers. If you want more features such as auto-registration, scanning, interceptors, or decorators then you may replace built-in IoC container with a third party container.
- ▶ The built-in container is represented by `IServiceProvider` implementation that supports constructor injection by default. The types (classes) managed by built-in IoC container are called services.
- ▶ **There are basically two types of services in ASP.NET Core:**
 1. Framework Services: Services which are a part of ASP.NET Core framework such as `IApplicationBuilder`, `IHostingEnvironment`, `ILoggerFactory` etc.
 2. Application Services: The services (custom types or classes) which you as a programmer create for your application.
- ▶ In order to let the IoC container automatically inject our application services, we first need to register them with IoC container.

.Net core - Dependency Injection

- ▶ **Registering Application Service:** Consider the following example of simple ILog interface and its implementation class. We will see how to register it with built-in IoC container and use it in our application.

```
public interface ILog
{
    void info(string str);
}

class MyConsoleLogger : ILog
{
    public void info(string str)
    {
        Console.WriteLine(str);
    }
}
```

- ▶ ASP.NET Core allows us to register our application services with IoC container, in the Program.cs there is a instance of WebApplicationBuilder and inside that there is Service property of type IServiceCollection, which is used to register application services.

.Net core - Dependency Injection

- ▶ Let's register ILogger with IoC container in Program.cs as shown below.

```
5 // Add services to the container.
6 builder.Services.AddRazorPages();
7 builder.Services.AddSingleton<ILogger, MyConsoleLogger>();
8
9 var app = builder.Build();
10
11 // Configure the HTTP request pipeline.
12 if (!app.Environment.IsDevelopment())
13 {
14     app.UseExceptionHandler("/Error");
15     // The default HSTS value is 30 days. You may want to change this for production scenarios
16     app.UseHsts();
17 }
18
19 app.UseMyMiddleware();
20 app.UseHttpsRedirection();
21 app.UseStaticFiles();
22 app.UseRouting();
23 app.UseAuthorization();
24 app.MapRazorPages();
25 app.Run();
```

.Net core - Dependency

Injection

- ▶ **Understanding Service Lifetime:** Built-in IoC container manages the lifetime of a registered service type. It automatically disposes a service instance based on the specified lifetime.
- ▶ The built-in IoC container supports three kinds of lifetimes:
 1. **Singleton:** IoC container will create and share a single instance of a service throughout the application's lifetime.
 2. **Transient:** The IoC container will create a new instance of the specified service type every time you ask for it.
 3. **Scoped:** IoC container will create an instance of the specified service type once per request and will be shared in a single request.
- ▶ **Constructor Injection:** Once we register a service, the IoC container automatically performs constructor injection if a service type is included as a parameter in a constructor.

.Net core - Dependency Injection

- ▶ For example, we can use ILog service type in any MVC controller. Consider the following example.

```
public class HomeController : Controller
{
    ILog _log;

    public HomeController(ILog log)
    {
        _log = log;
    }
    public IActionResult Index()
    {
        _log.info("Executing /home/index");

        return View();
    }
}
```

- ▶ **Action Method Injection:** In the example given on the right side, there might be situations where we only require the dependency service type within a specific action method. To achieve this, we can utilize the [FromServices] attribute along with the service type parameter in the method signature. This allows us to access the required service directly within that particular action method.

- ▶ In the provided example on the left, when working with an IoC container, the constructor of HomeController will be automatically provided with an instance of MyConsoleLogger. No additional manual intervention is required. Furthermore, the IoC container will handle the creation and disposal of an instance of ILog as per its registered lifetime.

```
using Microsoft.AspNetCore.Mvc;

public class HomeController : Controller
{
    public HomeController()
    {
    }

    public IActionResult Index([FromServices] ILog log)
    {
        log.info("Index method executing");

        return View();
    }
}
```

.Net core - Dependency Injection

- ▶ **Property Injection:** Built-in IoC container does not support property injection. You will have to use third party IoC container.
- ▶ **Get Services Manually:** It is not required to include dependency services in the constructor. We can access dependent services configured with built-in IoC container manually using RequestServices property of HttpContext as shown

```
public class HomeController : Controller
{
    public HomeController()
    {
    }
    public IActionResult Index()
    {
        var services = this.HttpContext.RequestServices;
        var log = (ILog)services.GetService(typeof(ILog));

        log.info("Index method executing");

        return View();
    }
}
```

- ▶ It is recommended to use constructor injection instead of getting it using RequestServices.

.Net core - Dependency Injection

- ▶ **Property Injection:** Built-in IoC container does not support property injection. You will have to use third party IoC container.
- ▶ **Get Services Manually:** It is not required to include dependency services in the constructor. We can access dependent services configured with built-in IoC container manually using RequestServices property of HttpContext as shown

```
public class HomeController : Controller
{
    public HomeController()
    {
    }
    public IActionResult Index()
    {
        var services = this.HttpContext.RequestServices;
        var log = (ILog)services.GetService(typeof(ILog));

        log.info("Index method executing");

        return View();
    }
}
```

- ▶ It is recommended to use constructor injection instead of getting it using RequestServices.

In this session

What we are learning in this session

- ▶ Introduction
- ▶ Create ASP.NET Core Web API

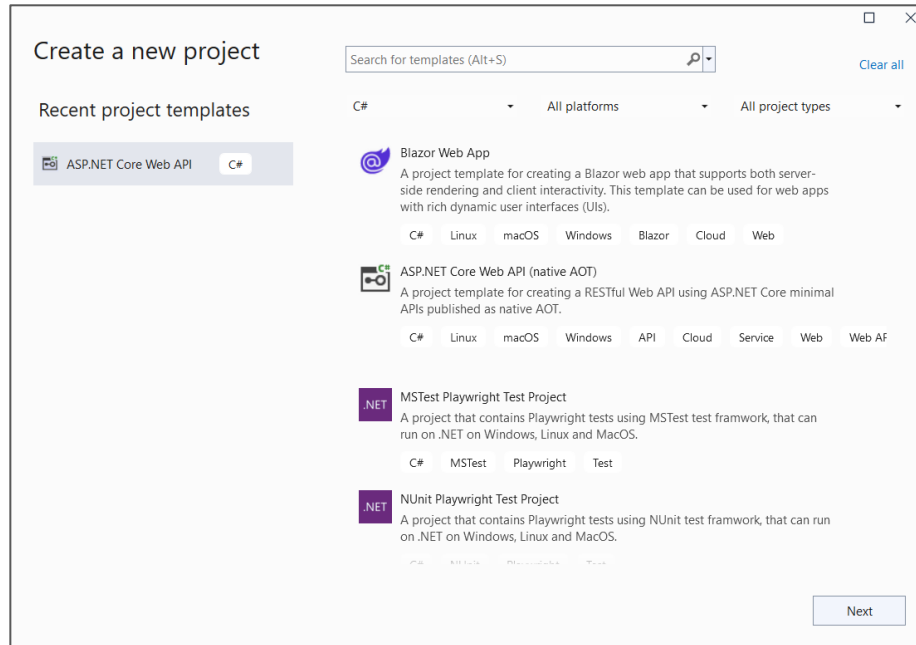


Introduction

- ▶ Authentication is the process of validating user credentials, and authorization is the process of checking privileges for a user to access specific modules in an application. In this article, we will see how to protect an ASP.NET Core Web API application by implementing JWT authentication.
- ▶ We will also see how to use authorization in ASP.NET Core to supply access to various functionalities of the application. We will store user credentials in an SQL server database, and we will use Entity Framework and Identity framework for database operations.
- ▶ JSON Web Token (JWT) is an open standard (RFC 7519) that defines a compact and self-contained way for securely transmitting information between parties as a JSON object. This information can be verified and trusted because it is digitally signed. JWTs can be signed using a secret (with the **HMAC** algorithm) or a public/private key pair using **RSA** or **ECDSA**.
- ▶ In its compact form, JSON Web Tokens consist of three parts separated by dots (.), which are:
 - Header
 - Payload
 - Signature
- ▶ Therefore, a JWT typically looks: **xxxx.yyyy.zzzz**

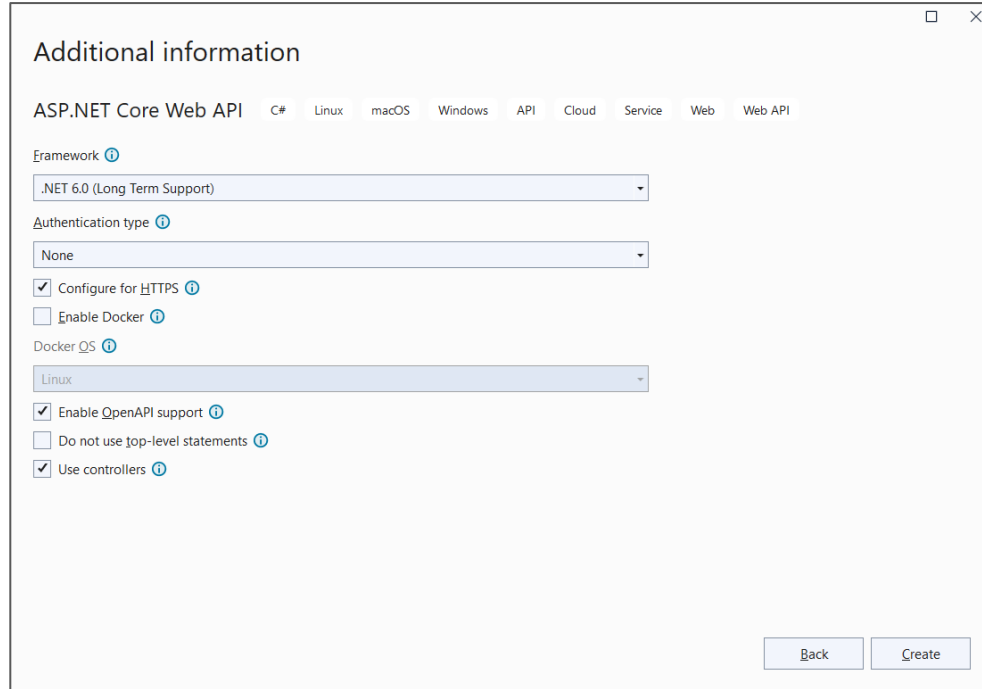
Create ASP.NET Core Web API

- ▶ We need Visual Studio 2022 to create .NET 6.0 applications. We can choose the ASP.NET Core Web API template from Visual Studio 2022.



Create ASP.NET Core Web API

- ▶ We can give a suitable name for our project and choose the .NET 6.0 framework.



The screenshot shows the 'Additional information' dialog box in Visual Studio. The title bar includes a maximize button and a close button. The main title is 'Additional information'. Below the title, there is a horizontal tab bar with the following tabs: 'ASP.NET Core Web API' (selected), 'C#', 'Linux', 'macOS', 'Windows', 'API', 'Cloud', 'Service', 'Web', and 'Web API'. The 'Framework' section has a dropdown menu set to '.NET 6.0 (Long Term Support)'. The 'Authentication type' section has a dropdown menu set to 'None'. There are three checkboxes: 'Configure for HTTPS' (checked), 'Enable Docker' (unchecked), and 'Docker OS' (set to 'Linux'). There are also three more checkboxes: 'Enable OpenAPI support' (checked), 'Do not use top-level statements' (unchecked), and 'Use controllers' (checked). At the bottom right, there are two buttons: 'Back' and 'Create'.

Additional information

ASP.NET Core Web API C# Linux macOS Windows API Cloud Service Web Web API

Framework ⓘ

.NET 6.0 (Long Term Support)

Authentication type ⓘ

None

☒ Configure for HTTPS ⓘ

☐ Enable Docker ⓘ

Docker OS ⓘ

Linux

☒ Enable OpenAPI support ⓘ

☐ Do not use top-level statements ⓘ

☒ Use controllers ⓘ

Back Create

Create ASP.NET Core Web API

- ▶ Our new project will be created in a few moments.
- ▶ We must install the 4 libraries below into the new project.
 - Microsoft.EntityFrameworkCore.SqlServer
 - Microsoft.EntityFrameworkCore.Tools
 - Microsoft.AspNetCore.Identity.EntityFrameworkCore
 - Microsoft.AspNetCore.Authentication.JwtBearer
- ▶ You can use the NuGet package manager to install the above packages.
- ▶ We can change the appsettings.json with values below. It has database connection details and other details for JWT authentication.

Create ASP.NET Core Web API

► appsettings.json

```
1  {
2    "Logging": {
3      "LogLevel": {
4        "Default": "Information",
5        "Microsoft.AspNetCore": "Warning"
6      }
7    },
8    "AllowedHosts": "*",
9    "ConnectionStrings": {
10     "ConnStr": "Data Source=(localdb)\\MSSQLLocalDB;Initial Catalog=JWTAuthDB;Integrated
11       Security=True;ApplicationIntent=ReadWrite;MultiSubnetFailover=False"
12   },
13   "JWT": {
14     "ValidAudience": "http://localhost:4200",
15     "ValidIssuer": "http://localhost:5000",
16     "Secret": "JWTAuthenticationHIGHsecuredPasswordVVVp10H7Xzyr"
17   }
18 }
```

Create ASP.NET Core Web API

- ▶ We can create a new folder, “Auth” and create the “ApplicationDbContext” class under Auth folder and add below code. We will add all the classes related to authentication under the Auth folder.
- ▶ **ApplicationDbContext.cs**

```
1  using Microsoft.AspNetCore.Identity;
2  using Microsoft.AspNetCore.Identity.EntityFrameworkCore;
3  using Microsoft.EntityFrameworkCore;
4
5  namespace JWTAuthentication.NET6._0.Auth
6  {
7      6 references
8      public class ApplicationDbContext : IdentityDbContext<IdentityUser>
9      {
10         0 references
11         public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options) : base(options)
12         {
13             0 references
14             protected override void OnModelCreating(ModelBuilder builder)
15             {
16                 base.OnModelCreating(builder);
17             }
18         }
19     }
```

Create ASP.NET Core Web API

- ▶ Create a static class “UserRoles” and add below values.
- ▶ UserRoles.cs

```
1 namespace JWTAuthentication.NET6._0.Auth
2 {
3     9 references
4     public static class UserRoles
5     {
6         public const string Admin = "Admin";
7         public const string User = "User";
8     }
```

- ▶ We have added two constant values “Admin” and “User” as roles. You can add as many roles as you wish.

Create ASP.NET Core Web API

- ▶ Create class "RegisterModel" for new user registration.
- ▶ RegisterModel.cs

```
1  using System.ComponentModel.DataAnnotations;
2
3  namespace JWTAuthentication.NET6._0.Auth
4  {
5      2 references
6      public class RegisterModel
7      {
8          [Required(ErrorMessage = "User Name is required")]
9          4 references
10         public string? Username { get; set; }
11
12         [EmailAddress]
13         [Required(ErrorMessage = "Email is required")]
14         2 references
15         public string? Email { get; set; }
16
17         [Required(ErrorMessage = "Password is required")]
18         2 references
19         public string? Password { get; set; }
20     }
21 }
```

Create ASP.NET Core Web API

- ▶ Create class “LoginModel” for user login.
- ▶ LoginModel.cs

```
1  using System.ComponentModel.DataAnnotations;
2
3  namespace JWTAuthentication.NET6._0.Auth
4  {
5      1 reference
6      public class LoginModel
7      {
8          [Required(ErrorMessage = "User Name is required")]
9          1 reference
10         public string? Username { get; set; }
11
12         [Required(ErrorMessage = "Password is required")]
13         1 reference
14         public string? Password { get; set; }
15     }
16 }
```


Create ASP.NET Core Web API

- ▶ We can create a class “Response” for returning the response value after user registration and user login. It will also return error messages if the request fails.

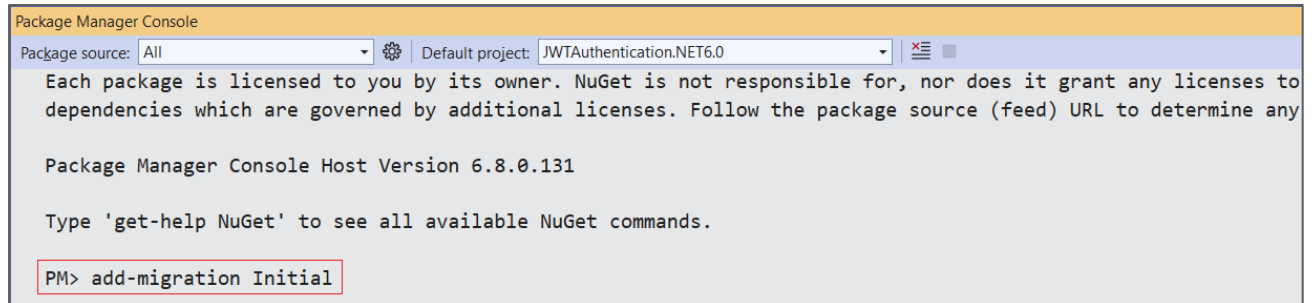
▶ Response.cs

```
1 namespace JWTAuthentication.NET6._0.Auth
2 {
3     6 references
4     public class Response
5     {
6         6 references
7         public string? Status { get; set; }
8         6 references
9         public string? Message { get; set; }
10    }
11 }
```

- ▶ We can create an API controller “AuthenticateController” inside the “Controllers” folder. Kindly check mentioned controller in Source Code.

Create ASP.NET Core Web API

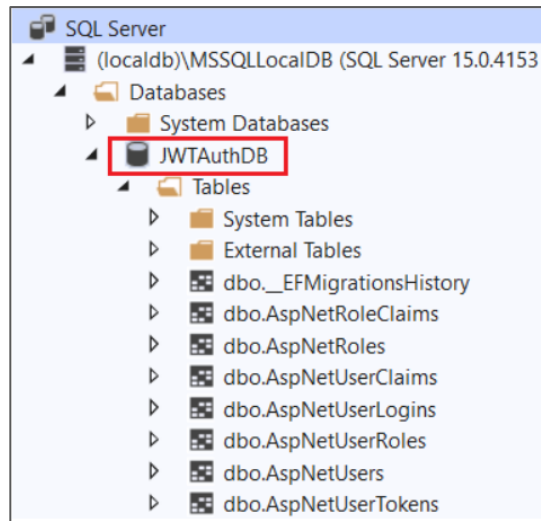
- ▶ We have added three methods “login”, “register”, and “register-admin” inside the controller class. Register and register-admin are almost same, but the register-admin method will be used to create a user with admin role. In login method, we have returned a JWT token after successful login.
- ▶ In .NET 6.0, Microsoft removed the Startup class and only kept Program class. We must define all our dependency injection and other configurations inside the Program class. Kindly check mentioned file in Source Code.
- ▶ We must create a database and tables needed before running the application. As we are using entity framework, we can use below database migration command with package manger console to create a migration script.
- ▶ **add-migration Initial**



```
Package Manager Console
Package source: All Default project: JWTAuthentication.NET6.0
Each package is licensed to you by its owner. NuGet is not responsible for, nor does it grant any licenses to dependencies which are governed by additional licenses. Follow the package source (feed) URL to determine any
Package Manager Console Host Version 6.8.0.131
Type 'get-help NuGet' to see all available NuGet commands.
PM> add-migration Initial
```

Create ASP.NET Core Web API

- ▶ Use the command below to create database and tables.
- ▶ **update-database**
- ▶ If you check the database using SQL server object explorer, you can see that the tables below are created inside the database.
- ▶ There are 7 tables created for User, Role and Claims during the database migration process. This is due to the Microsoft Identity framework.
- ▶ ASP.NET Core Identity is a membership system which allows you to add login functionality to your application. Users can create an account and login with a username and password, or they can use an external login provider such as Facebook, Google, Microsoft Account, Twitter and more.
- ▶ You can configure ASP.NET Core Identity to use a SQL Server database to store usernames, passwords, and profile data.



Create ASP.NET Core Web API

- ▶ We can add “Authorize” attribute inside the “WeatherForecast” controller.

```
[Authorize]
[ApiController]
[Route("api/[controller]")]
3 references
public class WeatherForecastController : ControllerBase
{
    private static readonly string[] Summaries = new[]
    {
        "Freezing", "Bracing", "Chilly", "Cool", "Mild", "Warm", "Balmy", "Hot", "Sweltering", "Scorching"
    };
};
```

- ▶ We can add “Authorize” attribute inside the “WeatherForecast” controller.

GET ⌵ http://localhost:5000/api/WeatherForecast

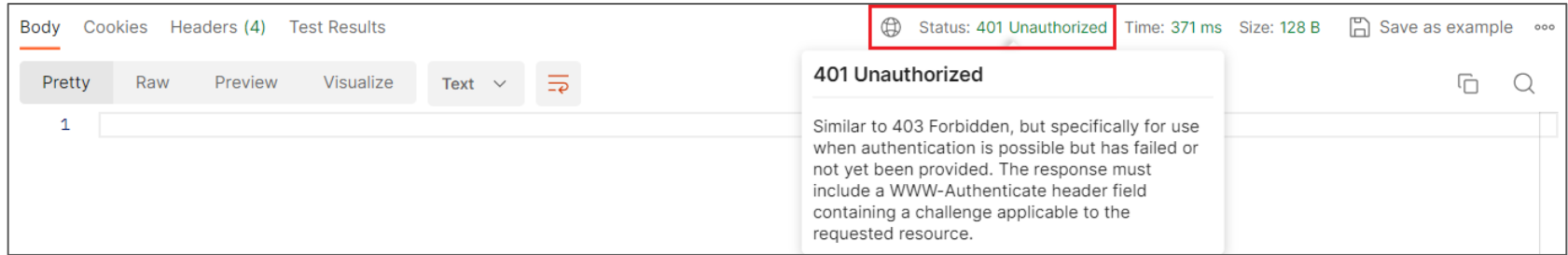
Params Authorization Headers (7) Body Pre-request Script Tests Settings

Query Params

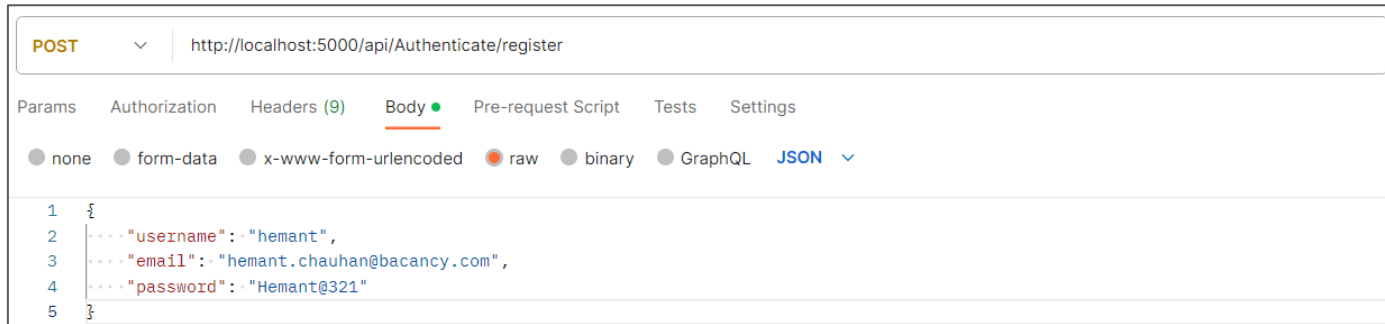
	Key	Value	Description
	Key	Value	Description

Create ASP.NET Core Web API

- ▶ We have received a 401 unauthorized error. Because, we have added Authorize attribute to entire controller. We must supply a valid token via request header to access this controller and methods inside the controller.



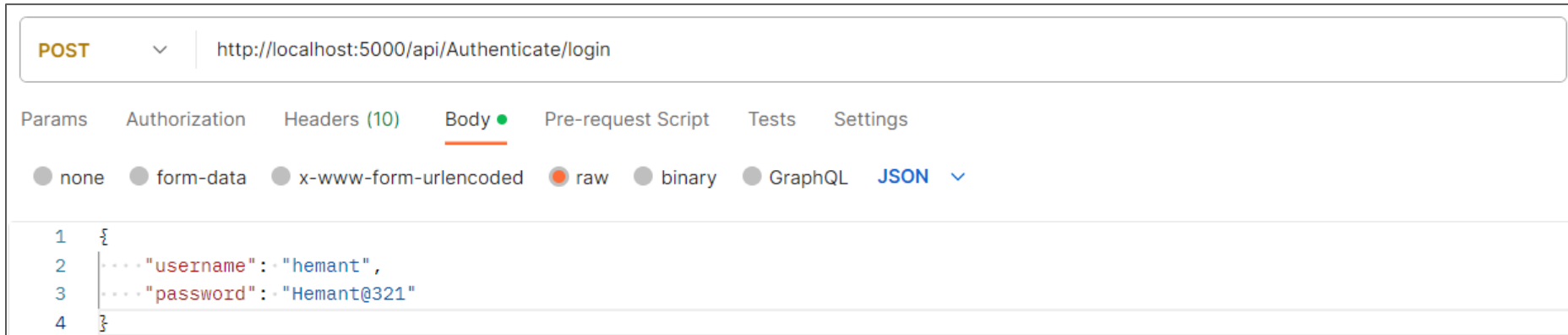
- ▶ We can create a new user using a register method in authenticate controller.



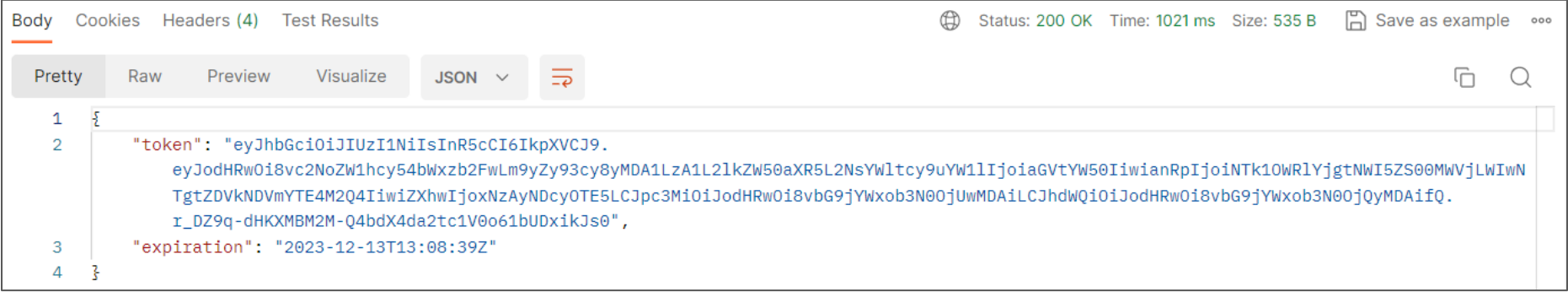
Create ASP.NET Core Web API



- ▶ We have supplied the input data as raw JSON format.
- ▶ We can use the above user credentials to login and get a valid JWT token.



Create ASP.NET Core Web API



- ▶ We have received a token after successful login with the above credentials.
- ▶ We can decode the token and see the claims and other information using <https://jwt.io> site.

Create ASP.NET Core Web API

- ▶ We can pass the above token value as a **Bearer** token inside the authorization tab and call get method of weatherforecast controller again.

The screenshot shows a REST client interface with the following details:

- Method:** GET
- URL:** `http://localhost:5000/api/WeatherForecast` (highlighted with a red box)
- Authorization:** Bearer Token (selected)
- Token:** `eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ...`
- Status:** 200 OK (highlighted with a red box)
- Time:** 45 ms
- Size:** 647 B
- Response (JSON):**

```
[
  {
    "date": "2023-12-14T15:48:19.5440854+05:30",
    "temperatureC": -18,
    "temperatureF": 0,
    "summary": "Chilly"
  }
]
```

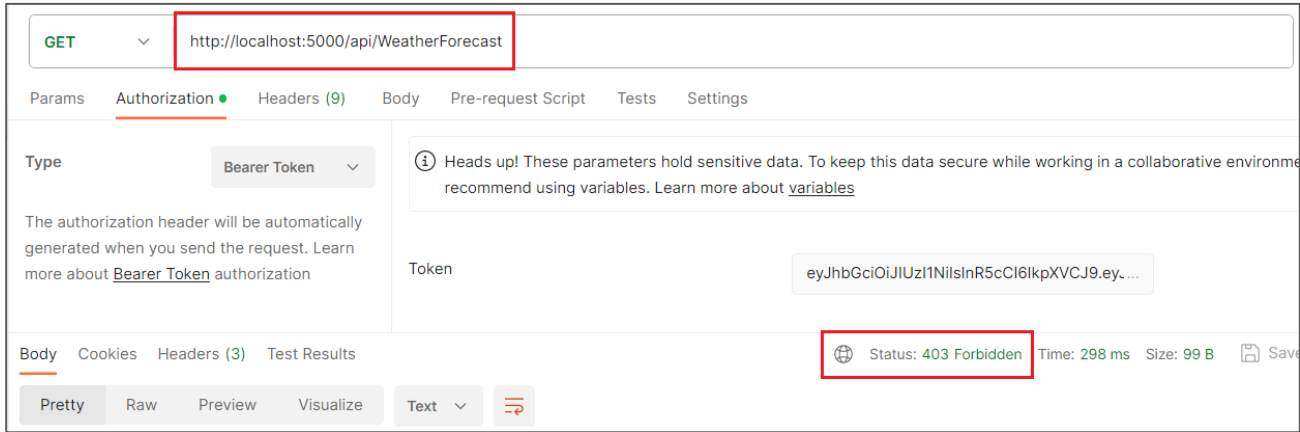
Create ASP.NET Core Web API

- ▶ This time, we have successfully received the values from the controller.
- ▶ We can change the weatherforecast controller with role-based authorization.

```
[Authorize(Roles = UserRoles.Admin)]  
[ApiController]  
[Route("api/[controller]")]  
3 references  
public class WeatherForecastController : ControllerBase  
{  
    private static readonly string[] Summaries = new[]  
    {  
        "Freezing", "Bracing", "Chilly", "Cool", "Mild", "Warm", "Balmy", "Hot", "Sweltering", "Scorching"  
    };  
};
```

- ▶ Now, only users with admin role can access this controller and methods.
- ▶ We can try to access the weatherforecast controller with same token again in Postman tool.

Create ASP.NET Core Web API



- ▶ We have received a 403 forbidden error instead of 401 now. Even though we are passing a valid token we don't have sufficient privilege to access the controller. To access this controller, the user must have an admin role permission. Current user is a normal user and does not have any admin role permission.
- ▶ We can create a new user with an admin role permission. We already have a method "register-admin" in authenticate controller for the same purpose.

Create ASP.NET Core Web API

The screenshot displays a REST client interface with the following details:

- Method:** POST
- URL:** `http://localhost:5000/api/Authenticate/register-admin` (highlighted with a red box)
- Body:** The request body is in JSON format, containing:

```
1 {  
2   "username": "hemant-admin",  
3   "email": "hemant.chauhan@bacancy.com",  
4   "password": "Hemant@321"  
5 }
```
- Response:** The response is also in JSON format, showing:

```
1 {  
2   "status": "Success",  
3   "message": "User created successfully!"  
4 }
```
- Status:** 200 OK (highlighted with a red box)
- Time:** 847 ms
- Size:** 207 B
- Buttons:** Pretty, Raw, Preview, Visualize, JSON (dropdown), and a Save icon.

- We can use this token instead of the old token to access the weatherforecast controller.

Create ASP.NET Core Web API

The screenshot shows a REST client interface with the following details:

- Method:** GET
- URL:** `http://localhost:5000/api/WeatherForecast` (highlighted with a red box)
- Authorization:** Bearer Token (selected)
- Token:** `eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ...`
- Status:** 200 OK (highlighted with a red box)
- Time:** 34 ms
- Size:** 648 B
- Body (JSON):**

```
1 [
2   {
3     "date": "2023-12-14T17:28:55.7140675+05:30",
4     "temperatureC": 32,
5     "temperatureF": 89,
6     "summary": "Balmy"
7   },
8 ]
```

- ▶ Now we have successfully fetched the data from weatherforecast controller.

SQL Server - Introduction

- ▶ SQL Server is a relational database management system, or RDBMS, developed and marketed by Microsoft.
- ▶ Similar to other RDBMS software, SQL Server is built on top of SQL, a standard programming language for interacting with relational databases.
- ▶ SQL Server is built for the basic function of storing retrieving data as required by other applications. It can be run either on the same computer or on another across a network.

SQL Server - Data Type

- ▶ In SQL Server, data type specifies the type of data that can be stored in a column of a table such as integer data, string data, date & time, binary strings, etc.
- ▶ Assigning an appropriate data type to columns in a table is crucial while designing a database. It affects the performance and efficiency of the database and the application using the database.

Category	Data Types
Exact numerics	bit, tinyint, smallint, int, bigint, decimal, numeric, money, smallmoney
Approximate numerics	Real, Float
Date & Time	date, smalldatetime, datetime, datetime2, datetimeoffset, time
Character strings	char, varchar, text
Unicode Character strings	nchar, nvarchar, ntext
Other data types	cursor, hierarchyid, sql_variant, spatial Geometry types, spatial Geography types, rowversion, uniqueidentifier, xml, table

- ▶ SQL Server provides built-in data types for all kinds of data that can be used within SQL Server.

SQL Server - Naming Conventions

- ▶ SQL Server defines a set of rules (dos and don'ts) for naming SQL Server Objects called naming convention, but also gives the user to follow their own preferred style. It is advisable to follow a single naming convention for all database objects consistently.
- ▶ **Why Use Naming Conventions?**
 - ❖ Naming convention for tables, columns, and all other related database objects like views, stored procedures, indexes, triggers, etc., are important for the success and maintenance of a project. A database can have many tables and users working on it.
 - ❖ By following a naming convention, you can spend less time finding what you need and helps in efficient database management.
- ▶ **SQL Server Database Object Naming Rules:**
 - ❖ Names of database objects such as tables, columns, stored procedures, views, etc, can contain alphanumeric characters and must begin with an alphabet or an underscore.
 - ❖ Case sensitivity is determined by the settings of the database in which the object is created. By default, SQL Server is not case sensitive.
 - ❖ A name that starts with the 'at' (@) sign always denotes a local variable or parameter and cannot be used as a name for any other type of object.

SQL Server - Naming Conventions

- ❖ A name that begins with the number sign (#) denotes a temporary table or procedure.
- ❖ The Object name should not be a SQL reserved word. SQL Server reserves both lowercase and uppercase versions of reserved words.
- ❖ Some system procedures and functions are given prefixes like 'sp_', 'xp_', or 'dt_' to signify that they are system objects and should be searched for in the master database. So it is better to avoid using these prefixes while creating database objects.
- ❖ There are a few commonly accepted styles to name a table & column in a database. It can be all 'lower case', 'UPPERCASE', 'Pascal Case' or 'camel Case'.
- ❖ Avoid using spaces between words for table names and columns as they will require square brackets and confuse scripts and procedures. Instead, use an underscore for joining multiple words. For example, EMPLOYEE_SKILL, employee_id instead of [EMPLOYEE ID].
- ❖ Avoid using prefixes like 'tbl_' for table names.
- ❖ While naming columns, use nouns that describe what is stored in the column.
 - Example: FIRST_NAME, ADDRESS, EMAIL, PHONE. Avoid using words that are just data types like text or date.

SQL Server - Tables

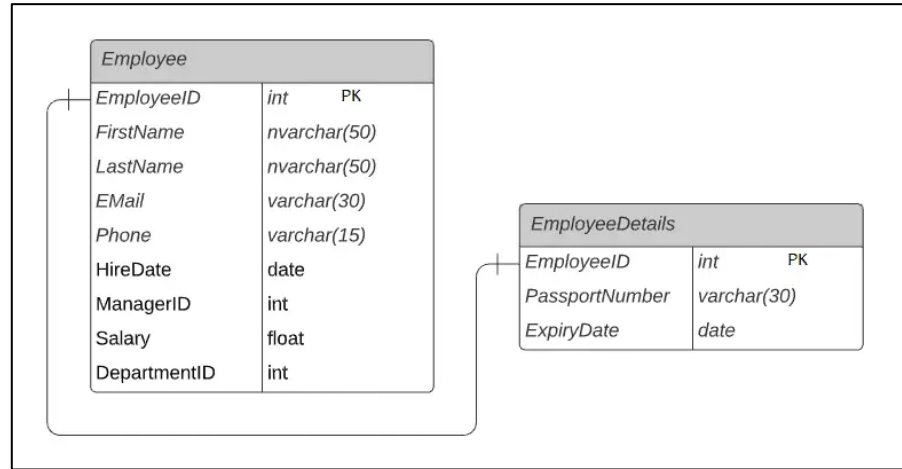
Relationships

- It is important to understand and design relationships among tables in a relational database like SQL Server. In a relational database, each table is connected to another table using the Primary-Foreign Key constraints.
- Table relationships in SQL Server database are of three types:** One-to-One, One-to-Many, and Many-to-Many

- 1). One-to-One:** In One-to-One relationship, one record of the first table will be linked to zero or one record of another table.

For example, each employee in the **Employee** table will have a corresponding row in **EmployeeDetails** table that stores the current passport details for that particular employee.

- Each employee will have zero or one record in the **EmployeeDetails** table. This is called zero or one-to-one relationship.



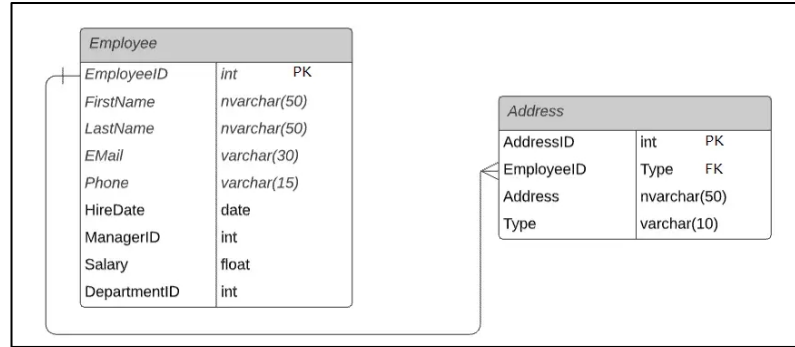
SQL Server - Tables

Relations

- ▶ Above, **EmployeeID** column is the primary key as well as foreign key column in the **EmployeeDetails** table that linked to **EmployeeID** of the **Employee** table. This forms zero or one-to-one relation.
- ▶ **2). One-to-Many Relation:** One-to-Many is the most commonly used relationship among tables. A single record from one table can be linked to zero or more rows in another table.
- ▶ Let's take an example of the **Employee** and **Address** table in the database. The **Employee** table stores employee records where **EmployeeID** is the primary key. The Address table holds the addresses of employees where **AddressID** is a primary key and **EmployeeID** is a foreign key. Each employee will have one record in the **Employee** table. Each employee can have many addresses such as Home address, Office Address, Permanent address, etc.
- ▶ The **Employee** and **Address** tables are linked by the key column **EmployeeID**. It is a foreign key in the Address table linking to the primary key **EmployeeID** in the **Employee** table. Thus, one record of the **Employee** table can point to multiple records in the Address table. This is a One-to-Many relationship.

SQL Server - Tables

Relations

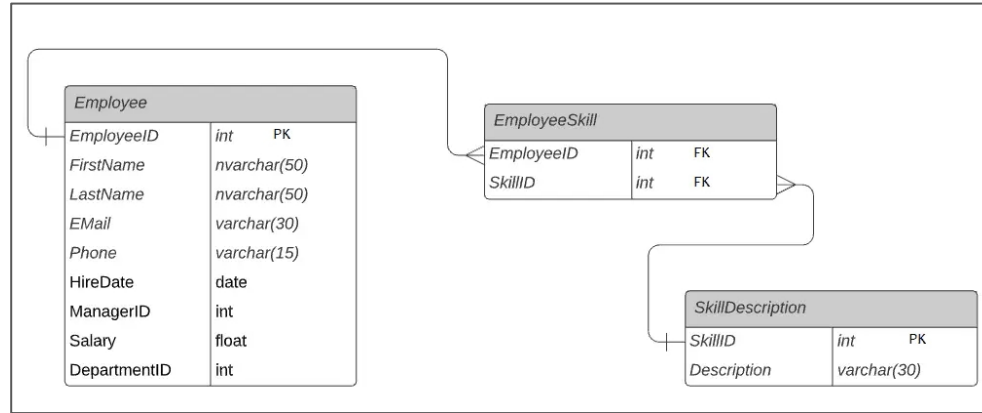


- ▶ **3). Many-to-Many Relation:** Many-to-Many relationship lets you relate each row in one table to many rows in another table and vice versa. As an example, an employee in the **Employee** table can have many skills from the **EmployeeSkill** table and also, one skill can be associated with one or more employees.
- ▶ The following figure demonstrates many-to-many relation between **Employee** and **SkillDescription** table using the junction table **EmployeeSkill**.

SQL Server - Tables

Relationships

- ▶ Every employee in the **Employee** table can have one or many skills. Similarly, a skill in the **SkillDescription** table can be linked to many employees. This makes a many-to-many relationship.



- ▶ In the example above, the **EmployeeSkill** is the junction table that contains **EmployeeID** and **SkillID** foreign key columns to form many-to-many relation between the **Employee** and **SkillDescription** table.

SQL Server -

Keys

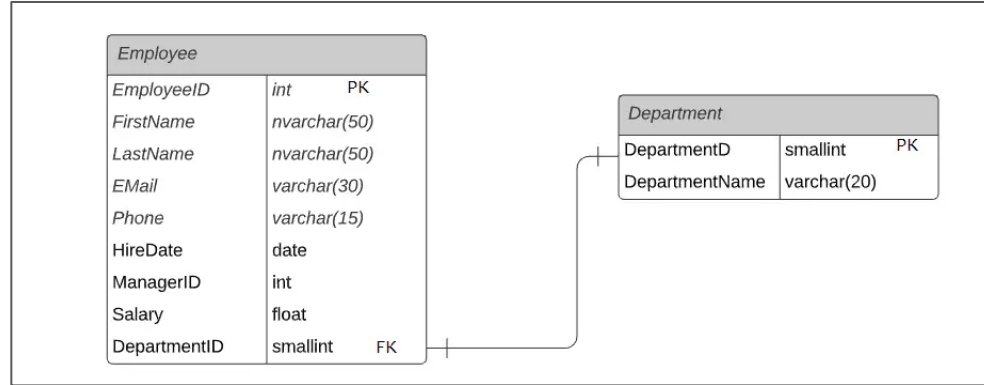
- ▶ **1). Primary Keys:** In SQL Server, a Primary key is a constraint that uniquely identify each row in the table. It enforce data integrity in tables.
 - A table can have only one primary key.
 - A primary key can be defined on one column or the combination of multiple columns known as a composite primary key.
 - The primary key uniquely identifies each row in a table. It is often defined on the identity column.
 - The Primary key column do not allow NULL or duplicate values. It will raise an error if try to do so.
 - All columns defined within the primary key constraint must be defined as a NOT NULL column.
 - If clustered or nonclustered is not specified, then a unique clustered index for the primary key column will be created if there no clustered index on the table. This makes retrieving data faster whenever the primary key column is included in the query.
- ▶ **Create a Primary Key:** A primary key can be configured in the create table SQL script. Append **CONSTRAINT** statement at the end of all column declaration.

SQL Server -

Keys

- ▶ **Syntax:** `CONSTRAINT <primarykey_name> PRIMARY KEY [CLUSTERED | NONCLUSTERED] (<column_names>)`
- ▶ The name of a primary key should be in PK_TableName_ColumnName format to recognize it easily.
- ▶ **Create a Primary Key in an Existing Table:** Use the **ALTER TABLE** statement to add a primary key in an existing table.
- ▶ **Syntax:** `ALTER TABLE Employee
ADD CONSTRAINT PK_Employee_EmployeeID PRIMARY KEY (EmployeeID)`
- ▶ **2). Foreign Keys:** The foreign key establishes the relationship between the two tables and enforces referential integrity in the SQL Server. For example, the following Employee table has a foreign key column DepartmentID that links to a primary key column of the Department table.
 - A foreign key column can be linked to a primary key or a unique key column of the same or another table.
 - The table having the foreign key constraint is called the child table, and the table being referenced by the foreign key is called the parent table. E.g. Employee is a child table and Department is a parent table.

SQL Server - Keys



- A value other than NULL is entered in the column of the foreign key constraint, that value must already exist in the referenced column of the parent table. Else you will get a foreign key violation error.
- A foreign key constraint defined at the table level (on a combination of columns) should have the same number of reference columns as the number of columns defined in the constraint list. The data type of each column in the constraint must be the same as the corresponding column in the column list.
- There is no limit on the number of foreign key constraints a table can contain that references other tables. However, it is limited by the hardware configuration and the database design.

SQL Server - Keys

- Foreign key constraints are not enforced on temporary tables.
- ▶ **3). Unique Keys:** The Unique Constraint ensures the uniqueness of all values and no duplicate values are entered in a column of a table.
- ▶ The unique constraints can be created using CREATE or ALTER TABLE SQL. Add unique constraints after columns definitions in the CREATE TABLE statement.
- ▶ **Syntax:** **CONSTRAINT** <constraint_name> **UNIQUE**(<column_name>)
- ▶ **Important Points:**
 - Both Unique constraint and Primary key constraint enforce uniqueness. It is recommended to use Unique constraint instead of Primary key constraint whenever you want to enforce uniqueness in a column.
 - Unlike Primary key constraint, Unique constraints allow only one NULL value.
 - A unique index is automatically created when a unique key constraint is created.
 - SQL Server raises an error whenever a duplicate value is inserted or updated.

SQL Server - Keys

▶ Important Points:

- When a Unique index is added to an existing column in a table, the database engine first checks for the uniqueness of the specified column data. If a duplicate entry is found, then the engine returns an error and does not add the constraint.
- A Unique key in a table can be referenced by a Foreign Key from another table.

In this course

What we are learning in this session

- ▶ Views
- ▶ Functions
- ▶ Stored Procedures
- ▶ Indexes
- ▶ Joins
- ▶ Create Database
- ▶ Create Table

SQL Server - Views

- ▶ In SQL Server, a view is a virtual table whose values are defined by a query. In another word, a view is a name given to a query that can be used as a table. The rows and columns of a view come from tables referenced by a query.

- ▶ **Types of Views:**

- 1). User-defined Views
- 2). Indexed Views
- 3). Partitioned Views
- 4). System Views

- ▶ **Syntax:**

```
CREATE VIEW <schema_name>.<view_name>  
AS  
SELECT column1, column2, ...  
FROM table1, table2,...  
[WHERE];
```

- ▶ **Important Points:**

- Unless indexed, a view does not exist as a stored set of data values in a database.
- Views can be created by using tables or other views in the current or other databases.
- The SQL statements comprising the view are stored in the database and not the resulting data.
- The data from a view is generated dynamically when a view is referenced.
- Views are used as a security mechanism to mask the underlying base tables and permit user access only to the view.

SQL Server - Functions

- ▶ Functions in SQL Server are similar to functions in other programming languages. Functions in SQL Server contains SQL statements that perform some specific tasks. Functions can have input parameters and must return a single value or multiple records.
- ▶ If your scripts use the same set of SQL statements repeatedly then this can be converted into a function in the database.
- ▶ **Types of Functions:**
SQL Server Functions are of two types.
 - 1). **System Functions:** These are built-in functions available in every database. Some common types are Aggregate functions, Analytic functions, Ranking functions, Rowset functions, Scalar functions.
 - 2). **User Defined Functions (UDFs):** Functions created by the database user are called User-defined functions. UDFs are of two types:

SQL Server - Functions

2.1). Scalar functions: The function that returns a single data value is called a scalar function.

2.2). Table-valued functions: The function that returns multiple records as a table data type is called a Table-valued function. It can be a result set of a single select statement.

▶ Advantage of User-defined Functions:

- **Faster Execution:** Similar to stored procedures, UDFs reduce the compilation cost of T-SQL by caching the plans and reusing them for future executions.
- **Reduce Network Traffic:** The SQL statements of a function execute in the database, and the application calling it needs to make a function call to the database.
- **Supports Modular Programming:** UDFs can be modified independently of the application source code. You can create UDFs once and store them in the database, and they can be called any number of times.

SQL Server - Stored Procedures

- ▶ In SQL Server, a stored procedure is a set of T-SQL statements which is compiled and stored in the database. The stored procedure accepts input and output parameters, executes the SQL statements, and returns a result set if any.
- ▶ By default, a stored procedure compiles when it gets executed for the first time. It also creates an execution plan that is reused for subsequent executions for faster performance.
- ▶ **Stored procedures are of two types:**
 - 1). **User-defined procedures:** A User-defined stored procedure is created by a database user in a user-defined database or any System database except the resource database.
 - 2). **System procedures:** System procedures are included with SQL Server and are physically stored in the internal, hidden Resource database and logically appear in the sys schema of all the databases. The system stored procedures start with the sp_ prefix.
- ▶ **Create Stored Procedure:**

Use the CREATE statement to create a stored procedure.

SQL Server - Stored Procedures

Syntax:

```
CREATE [OR ALTER] {PROC | PROCEDURE} [schema_name.] procedure_name([@parameter
data_type [ OUT | OUTPUT | [READONLY]] [ WITH <procedure_option> ]
[ FOR REPLICATION ]
AS
    BEGIN
        sql_statements
    END
```

Advantages of Stored procedures:

- Stored procedures are reusable. Multiple users in multiple applications can use the same Stored Procedure.
- SPs reside in the database, it reduces network traffic. Applications have to make a procedure call to the database and it communicates back to the user.

SQL Server - Stored Procedures

► Advantages of Stored procedures:

- Database objects are encapsulated within a stored procedure, and this acts as a security mechanism by restricting access to the database objects. Reduced development cost, easily modified, and increased readability.
- Improves performance. When a stored procedure is executed for the first time, the database processor creates an execution plan which is reused every time this SP is executed.

SQL Server - Indexes

- ▶ An Index in SQL Server is a data structure associated with tables and views that helps in faster retrieval of rows.
- ▶ Data in a table is stored in rows in an unordered structure called Heap. If you have to fetch data from a table, the query optimizer has to scan the entire table to retrieve the required row(s). If a table has a large number of rows, then SQL Server will take a long time to retrieve the required rows. So, to speed up data retrieval, SQL Server has a special data structure called indexes.
- ▶ An index is mostly created on one or more columns which are commonly used in the SELECT clause or WHERE clause.
- ▶ **There are two types of indexes in SQL Server:**
Clustered Indexes, and Non-Clustered Indexes
- ▶ **Clustered Indexes:**
The clustered index defines the order in which the table data will be sorted and stored. As mentioned before, a table without indexes will be stored in an unordered structure. When you define a clustered index on a column, it will sort data based on that column values and store it. Thus, it helps in faster retrieval of the data.

SQL Server - Indexes

- ▶ There can be only one clustered index on a table because the data rows can be stored in only one order.
- ▶ When you create a Primary Key constraint on a table, a unique clustered index is automatically created on the table.

▶ **Syntax:** `CREATE CLUSTERED INDEX <index_name>
 ON [schema.]<table_name>(column_name [asc|desc]);`

▶ Non-Clustered Indexes:

SQL Server provides two types of indexes, clustered and non-clustered indexes. Here you will learn non-clustered indexes.

- ▶ The non-clustered index does not sort the data rows physically. It creates a separate key-value structure from the table data where the key contains the column values (on which a non-clustered index is declared) and each value contains a pointer to the data row that contains the actual value. It is similar to a textbook having an index at the back of the book with page numbers pointing to the actual information.

SQL Server - Indexes

- ▶ There can be 999 non-clustered indexes on a single table is 999. When you create a Unique constraint, a unique non-clustered index is created on the table.

▶ **Syntax:** `CREATE NONCLUSTERED INDEX <index_name>
ON <table_name>(column)`

- ▶ When you create a Primary Key constraint on a table, a unique clustered index is automatically created on the table.

▶ **Syntax:** `CREATE CLUSTERED INDEX <index_name>
ON [schema.]<table_name>(column_name [asc|desc]);`

SQL Server - Joins

- ▶ SQL Server joins are essential for retrieving data from multiple tables in a relational database. They allow you to combine rows from two or more tables based on a related column between them.
- ▶ **1). INNER JOIN:** An INNER JOIN returns only the rows that have matching values in both tables.

```
SELECT column1, column2, ...  
FROM table1  
INNER JOIN table2  
ON table1.column_name = table2.column_name;
```

- ▶ **2). LEFT JOIN (or LEFT OUTER JOIN):** A LEFT JOIN returns all rows from the left table and the matched rows from the right table. If there's no match, it returns NULL for columns from the right table.

```
SELECT column1, column2, ...  
FROM table1  
LEFT JOIN table2  
ON table1.column_name = table2.column_name;
```

SQL Server - Joins

- ▶ **3). RIGHT JOIN (or RIGHT OUTER JOIN):** A RIGHT JOIN returns all rows from the right table and the matched rows from the left table. If there's no match, it returns NULL for columns from the left table.

```
SELECT column1, column2, ...  
FROM table1  
RIGHT JOIN table2  
ON table1.column_name = table2.column_name;
```

- ▶ **4). FULL OUTER JOIN:** A FULL OUTER JOIN returns all rows when there is a match in either the left or the right table. If there's no match, it returns NULL for columns from the table without a match.

```
SELECT column1, column2, ...  
FROM table1  
FULL OUTER JOIN table2  
ON table1.column_name = table2.column_name;
```

SQL Server - Joins

- ▶ **5). SELF JOIN:** A SELF JOIN is used to join a table with itself. It's often used when you have hierarchical or parent-child relationships within a table.

```
SELECT column1, column2, ...  
FROM table1 AS t1  
INNER JOIN table1 AS t2  
ON t1.column_name = t2.column_name;
```

- ▶ **6). CROSS JOIN:** A CROSS JOIN returns the Cartesian product of two tables, meaning it combines every row from the first table with every row from the second table.

```
SELECT column1, column2, ...  
FROM table1  
CROSS JOIN table2;
```

Create Database

- ▶ In SQL Server, a database is made up of a collection of objects like tables, functions, stored procedures, views etc. Each instance of SQL Server can have one or more databases. SQL Server databases are stored in the file system as files.
- ▶ A login is used to gain access to a SQL Server instance and a database user is used to access a database. SQL Server Management Studio is widely used to work with a SQL Server database.
- ▶ **Type of Database in SQL Server**
There are two types of databases in SQL Server: **System Database** and **User Database**.
- ▶ **System databases** are created automatically when SQL Server is installed. They are used by SSMS and other SQL Server APIs and tools, so it is not recommended to modify the system databases manually.

The followings are the system databases:

- master: master database stores all system level information for an instance of SQL Server. It includes instance-wide metadata such as logon accounts, endpoints, linked servers, and system configuration settings.

Create Database

▶ **The followings are the system databases:**

- **model:** model database is used as a template for all databases created on the instance of SQL Server
- **msdb:** msdb database is used by SQL Server Agent for scheduling alerts and jobs and by other features such as SQL Server Management Studio, Service Broker and Database Mail.
- **tempdb:** tempdb database is used to hold temporary objects, intermediate result sets, and

▶ **User-defined Databases** are created by the database user using T-SQL or SSMS for your application data. A maximum of 32767 databases can be created in an SQL Server instance.

▶ You can execute the SQL script in the query editor using Master database.

▶ **Syntax:** **USE** master;
 CREATE <database-name>

Create Table

- ▶ Tables are database objects that contain all the data in a database. In a table, data is logically organized in rows and columns. Each column represents a field and each row represents a unique record. Each column has a data type associated with it. It represents the type of data in that column. Each table is uniquely named in a database.
- ▶ Number of tables in a database is only limited by the number of objects allowed in the database (2,147,483,647). A user-defined table can have up to 1024 columns.
- ▶ You can execute the CREATE TABLE statement in the query editor of SSMS to create a new table in SQL Server.

▶ **Syntax:** `CREATE TABLE [database_name.][schema_name.]table_name (
 pk_column_name data_type PRIMARY KEY,
 column_name2 data_type [NULL | NOT NULL],
 column_name3 data_type [NULL | NOT NULL],
 ...,
 [table_constraints]
);`

Create Table

- ▶ Tables are database objects that contain all the data in a database. In a table, data is logically organized in rows and columns. Each column represents a field and each row represents a unique record. Each column has a data type associated with it. It represents the type of data in that column. Each table is uniquely named in a database.

In this course

What we are learning in this session

- ▶ Transactions - TCL Command
- ▶ Error Handling in SQL
- ▶ CTE
- ▶ Temp Table
- ▶ Variables
- ▶ Constraints
- ▶ Clustered vs Nonclustered
- ▶ Trigger
- ▶ Cursor

SQL Command

- SQL commands are instructions. It is used to communicate with the database. It is also used to perform specific tasks, functions, and queries of data.

1. DDL (Data Definition Language)
2. DML (Data Manipulation Language)
3. DCL (Data Control Language)
4. TCL (Transaction Control Language)
5. DQL(Data Query Language)

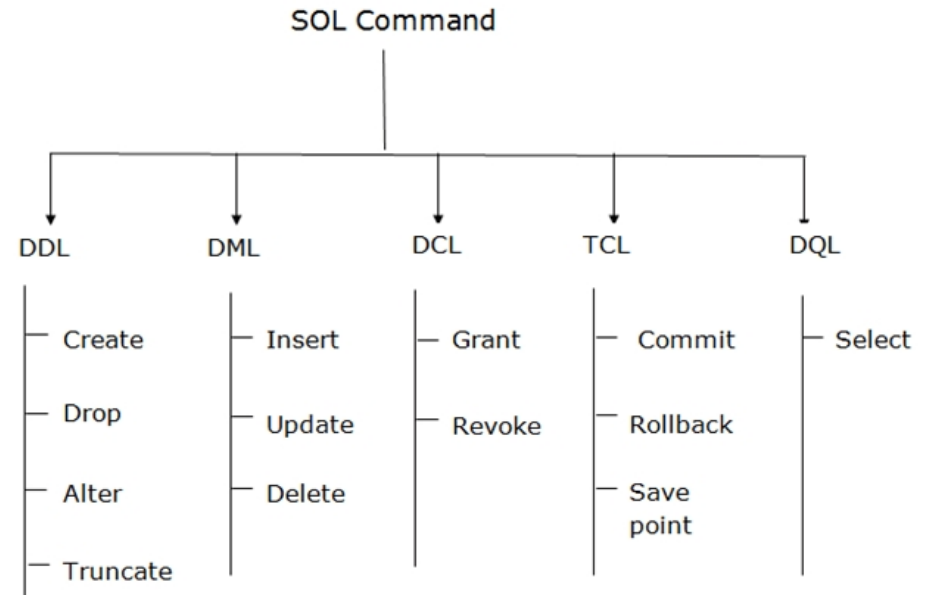


Table - DDL Command



DDL Command

- ▶ In SQL, a table is a structured representation of data organized into rows and columns. It is a fundamental database object used to store, manage, and retrieve information.

- ▶ **Create Table :**

```
CREATE TABLE employees (  
    employee_id INT PRIMARY KEY,  
    employee_name VARCHAR(50),  
    department VARCHAR(30),  
    salary DECIMAL(10, 2)  
)
```

- ▶ **Alter Table:**

```
ALTER TABLE employees ADD hire_date DATE
```

DDL Command

- ▶ **Drop Table:**

DROP TABLE employees

- ▶ DROP is used to delete database objects, such as tables, views, indexes, or databases themselves.

- ▶ **Truncate Table:**

TRUNCATE TABLE employees

- ▶ TRUNCATE is used to remove all records from a table but retains the table structure for future use. When we use TRUNCATE it will reset column identity.

Syntax:

DELETE FROM your_table WHERE condition

TRUNCATE TABLE your_table

- ▶ DELETE is a row-by-row operation allowing conditional removal, while TRUNCATE efficiently removes all rows in one operation without conditions but may not be suitable if foreign key constraints are present or if you need to log individual deletions.

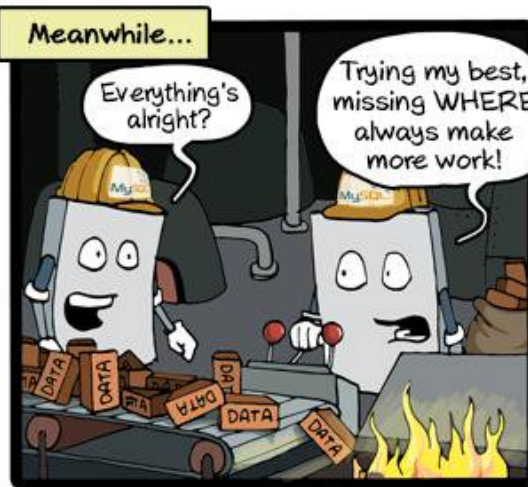
DQL & DML Command

DQL & DML

- ▶ DML (Data Manipulation Language) commands in SQL Server are used to interact with and manipulate data stored in tables.
 - `SELECT EmployeeID, FirstName, LastName, Salary, DepartmentID
FROM Employee
WHERE DepartmentID = 1`
 - `INSERT INTO Employee (FirstName, LastName, Salary, DepartmentID)
VALUES ('John', 'Doe', 50000, 2)`
 - `UPDATE Employee
SET Salary = 55000
WHERE EmployeeID = 101`
 - `DELETE FROM Employee
WHERE EmployeeID = 102`

DQL & DML

- ORDER BY : Retrieve all employees sorted by their salary in descending order. Default order type is ascending.
 - SELECT * FROM Employee
ORDER BY Salary DESC
- GROUP BY : Count the number of employees in each department
 - SELECT Department, COUNT(*) AS EmployeeCount
FROM Employee
GROUP BY Department
- DISTINCT : Retrieve unique departments from the Employee table
 - SELECT DISTINCT Department
FROM Employee
- SUM, AVG, MIN, MAX
 - SELECT SUM(Salary) AS TotalSalary, AVG(Salary) AS AvgSalary,
MIN(Salary) AS MinSalary, MAX(Salary) AS MaxSalary
FROM Employee



DCL Command

DCL

- ▶ DCL (Data Control Language) commands in SQL are used to control access to the data within a database.
 - GRANT SELECT, INSERT ON Employee TO user1
 - REVOKE DELETE ON Employee FROM user2

TCL Command

TCL

- ▶ TCL (Transaction Control Language) commands in SQL are used to manage transactions.
- ▶
 1. COMMIT: Saves all the changes made during the current transaction to the database.
 2. ROLLBACK: Undoes all changes made during the current transaction, reverting the database to its state before the transaction started.
 3. SAVEPOINT: Sets a named point within a transaction to which you can later roll back.
 4. ROLLBACK TO SAVEPOINT: Rolls back the transaction to a specified savepoint.

```
BEGIN TRANSACTION;
```

```
INSERT INTO Employee (EmployeeID, FirstName, LastName, Salary, DepartmentID) VALUES (101, 'John', 'Doe', 50000, 1);
```

```
UPDATE Employee SET Salary = 55000 WHERE EmployeeID = 101;
```

```
SAVEPOINT my_savepoint;
```

```
INSERT INTO Employee (EmployeeID, FirstName, LastName, Salary, DepartmentID) VALUES (102, 'Jane', 'Smith', 60000, 3);
```

```
ROLLBACK TO SAVEPOINT my_savepoint;
```

```
COMMIT;
```


Error Handling

Error Handling in SQL

- ▶ Error handling in SQL Server gives us control over the Transact-SQL code. For example, when things go wrong, we get a chance to do something about it and possibly make it right again.
- ▶ Why Error Handling?

Prevents unexpected failures

Helps in debugging & troubleshooting

Ensures data consistency

Improves user experience

- ▶ Types of Errors:

Syntax Errors - Incorrect SQL syntax

Runtime Errors - Division by zero, type mismatches

Logical Errors - Incorrect business logic

Error Handling in SQL

▶ Error Handling Methods:

- 1 **TRY...CATCH** - Handles runtime errors
- 2 **RAISEERROR** & **THROW** - Raises custom errors
- 3 **@@ERROR** - Retrieves error codes

▶ TRY...CATCH Block (Example):

```
BEGIN TRY
    SELECT 1/0 -- Division by zero error
END TRY
BEGIN CATCH
    PRINT 'Error: ' + ERROR_MESSAGE()
END CATCH
```

CTE (Common Table Expression)

CTE

- ▶ A Common Table Expression is a temporary result set that you can reference within a SELECT, INSERT, UPDATE, or DELETE statement
- ▶ CTEs are particularly useful for making complex queries more readable and maintainable by breaking them down into smaller, logical units. They can also be recursive, allowing you to perform hierarchical queries.
- ▶ WITH SalesCTE (Product, TotalSales) AS (
 SELECT Product, SUM(SalesAmount) AS TotalSales
 FROM Sales
 GROUP BY Product)
SELECT * FROM SalesCTE

CTE

► Benefits :

1. CTEs make code more readable. And readability makes queries easier to debug.
2. CTEs can reference the results multiple times throughout the query. By storing the results of the subquery, you can reuse them throughout a larger query.
3. CTEs can help you perform multi-level aggregations. Use CTEs to store the results of aggregations, which you can then summarize in the main query.

Reference

- <https://stackoverflow.com/questions/54176/what-are-the-advantages-disadvantages-of-using-a-cte>

Temp Table

Temp Table

- ▶ In SQL Server, a temporary table is a table that is created and exists only for the duration of a session or a specific scope of work. Temporary tables are particularly useful when you need to store and manipulate intermediate results within a specific operation, and you don't want to clutter the permanent database with such temporary data.
- ▶ Use Cases for Temporary Tables:
 1. **Complex Queries:** Temporary tables can be used to store intermediate results during complex queries, improving readability and performance.
 2. **Transaction Isolation:** Temporary tables can help isolate data within a transaction, preventing interference with other transactions.
 3. **Reusability:** Temporary tables allow you to reuse the same set of data within a session without having to recreate the table structure.
 4. **Temporary Storage:** Storing data temporarily for a specific process or reporting task.

Temp Table

► Types of Temp Table :

1. Local Temporary Table :

- Local temp tables are similar to Permanent Tables in SQL Server, it accepts the single hash value "#" as the prefix when created. They are visible only to the connection that creates it, and are deleted when the connection is closed.
- ```
CREATE TABLE #TempTable (
 ID INT,
 Name VARCHAR(255)
)
```
- ```
INSERT INTO #TempTable VALUES (1, 'John'), (2, 'Jane')
```
- ```
SELECT * FROM #TempTable
```
- ```
DROP IF EXISTS TABLE #TempTable
```
- ```
SELECT * INTO #TempTable FROM Employee
```

# Temp Table

- ▶ 2. Global Temporary Table
  - Global Temporary Tables are also similar to Local Temporary Tables in SQL Server, except two "##" values are used as the prefix at the time of their creation. They are visible to all connections of SQLServer and only destroyed when the last connection referencing the table is closed (in which we have created the Global Temporary Table).
  - CREATE TABLE ##GlobalTempTable (  
    ID INT,  
    Name VARCHAR(255)  
)
  - INSERT INTO ##GlobalTempTable VALUES (1, 'Mike'), (2, 'Sara')
  - SELECT \* FROM ##GlobalTempTable
  - DROP IF EXISTS TABLE ##GlobalTempTable
  - SELECT \* INTO ##GlobalTempTable FROM Employee

# Variables

# Variable

- ▶ Variable is a storage location with a name that you assign to it. It is used to store and manipulate data temporarily within the scope of a batch, stored procedure, function, or trigger. Variables provide a way to pass data between different parts of a script or stored procedure, making the code more flexible and dynamic.
- ▶ `DECLARE @MyVariable INT`  
`SET @MyVariable = 42`

# Variable

|                  | Temporary Tables                                                                                          | Variables                                                                                                              |
|------------------|-----------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| Scope            | Temporary tables are visible across sessions and can be used by multiple queries within the same session. | Variables have a narrower scope and are typically limited to the batch or stored procedure in which they are declared. |
| Data Persistence | Data in temporary tables persists until the session ends or the table is explicitly dropped.              | Data stored in variables only persists for the duration of the batch or procedure execution.                           |
| Performance      | using temporary tables might introduce additional overhead, as they involve disk I/O (writing to tempdb). | variables have lower overhead as they are stored in memory.                                                            |

# Constraints

# Constraints

- ▶ Constraints in SQL Server are predefined rules and restrictions that are enforced in a single column or multiple columns, regarding the values allowed in the columns, to maintain the integrity, accuracy, and reliability of that column's data.
- ▶ **Types of Constraints:**
  - 1) PRIMARY KEY - Ensures unique, non-null values in a column
  - 2) FOREIGN KEY - Maintains referential integrity between tables
  - 3) UNIQUE - Ensures all values in a column are unique
  - 4) CHECK - Enforces a specific condition on column values
  - 5) DEFAULT - Assigns a default value if none is provided
  - 6) NOT NULL - Ensures a column cannot store NULL values

# Clustered vs Non-Clustered



# Clustered vs Non clustered

- ▶ What is an Index?
  - Improves query performance by organizing data efficiently
  - Two main types: **Clustered** and **Non-Clustered**

| Feature          | Clustered Index                       | Non-Clustered Index                        |
|------------------|---------------------------------------|--------------------------------------------|
| Data Storage     | Stores actual data in sorted order    | Stores pointers to actual data             |
| Number per Table | Only 1 per table                      | Multiple allowed                           |
| Speed            | Faster for SELECT queries             | Slower than clustered for large data       |
| Usage            | Ideal for primary keys                | Useful for search columns                  |
| Example          | PRIMARY KEY creates a clustered index | CREATE INDEX creates a non-clustered index |

- ▶ -- Clustered Index (Automatically created on PK) ▶ -- Non-Clustered Index

```
CREATE TABLE Employees (
 EmployeeID INT PRIMARY KEY,
 Name NVARCHAR(50)
);

CREATE NONCLUSTERED
INDEX IX_Employees_Name
ON Employees(Name);
```

Trigger

# Trigger

- ▶ A trigger is a special type of stored procedure that automatically runs when an event occurs in the database server. DML triggers run when a user tries to modify data through a data manipulation language (DML) event. DML events are INSERT, UPDATE, or DELETE statements on a table or view.
- ▶ DDL triggers run in response to a variety of data definition language (DDL) events. These events primarily correspond to Transact-SQL CREATE, ALTER, and DROP statements
- ▶ Reference  
<https://www.sqlshack.com/triggers-in-sql-server/>  
<https://learn.microsoft.com/en-us/sql/t-sql/statements/create-trigger-transact-sql?view=sql-server-ver16>

# Trigger

## ► Advantages of Triggers

1. **Enforcing Business Rules:** Triggers can be used to enforce business rules by automatically checking and validating data before it is inserted, updated, or deleted. This helps maintain data integrity.
2. **Audit Log:** Triggers can be employed to maintain an audit log of changes, tracking who made the changes and when. This is beneficial for compliance and debugging purposes.
3. **Synchronization of Data:** Triggers can be used to synchronize data across different tables or databases when changes are made, ensuring consistency.

## ► Disadvantages or Drawbacks of Triggers

1. **Performance Impact:** Triggers can introduce performance overhead because they add additional processing during data modification operations.
2. **Hidden Logic:** Triggers contain logic that is not immediately evident when examining the SQL statements for a given operation. This can make it challenging to understand and debug the overall system logic.

Cursor

# Cursor

- ▶ Cursor is a database object that allows us to iterate over a result set, row by row, and perform operations on each row individually. Cursors are typically used within Transact-SQL (T-SQL) scripts or stored procedures to process data sequentially.

1. Forward-Only Cursor
2. Static Cursor
3. Dynamic Cursor
4. Keyset-Driven Cursor
5. Scrollable Cursor

- ▶ Reference

<https://learn.microsoft.com/en-us/sql/relational-databases/cursors?view=sql-server-ver16>

# Cursor

## ▶ Advantages of Cursors

1. **Explicit Control:** Cursors provide explicit control over the fetching, updating, and deleting of rows. This can be useful in situations where we need fine-grained control over data manipulation.
2. **Scrolling:** Cursors provide the ability to move back and forth through the result set, allowing for more flexible navigation compared to a simple query.

## ▶ Disadvantages of Cursors

1. **Performance Overhead:** Cursors can result in poor performance, especially when processing large result sets, as they may require additional resources and increase database load.
2. **Blocking:** Cursors can lead to increased locking and blocking issues, especially in concurrent environments, where other transactions may be affected by the locks held by the cursor.
3. **Complexity:** Code using cursors tends to be more complex and harder to maintain than set-based operations. SQL is designed to work with sets of data efficiently, and using cursors might deviate from this paradigm.

Thank you



# DEDICATED TO INNOVATION

---

[www.bacancy.com](http://www.bacancy.com)





**Customer Satisfaction  
Is Our Highest Priority**



# Employee Matters



# In this session

## What we are learning in this session

- ▶ Introduction of EF
- ▶ EF Features
- ▶ EF Architecture
- ▶ EF Installation
- ▶ Data Annotation

Let's Start

# Entity Framework - Introduction

- ▶ Prior to .NET 3.5, we (developers) often used to write ADO.NET code or Enterprise Data Access Block to save or retrieve application data from the underlying database. We used to open a connection to the database, create a DataSet to fetch or submit the data to the database, convert data from the DataSet to .NET objects or vice-versa to apply business rules. This was a cumbersome and error prone process. Microsoft has provided a framework called "Entity Framework" to automate all these database related activities for your application.
- ▶ Entity Framework is an open-source ORM framework for .NET applications supported by Microsoft. It enables developers to work with data using objects of domain specific classes without focusing on the underlying database tables and columns where this data is stored.
- ▶ With the Entity Framework, developers can work at a higher level of abstraction when they deal with data, and can create and maintain data-oriented applications with less code compared with traditional applications.

# EF - Features

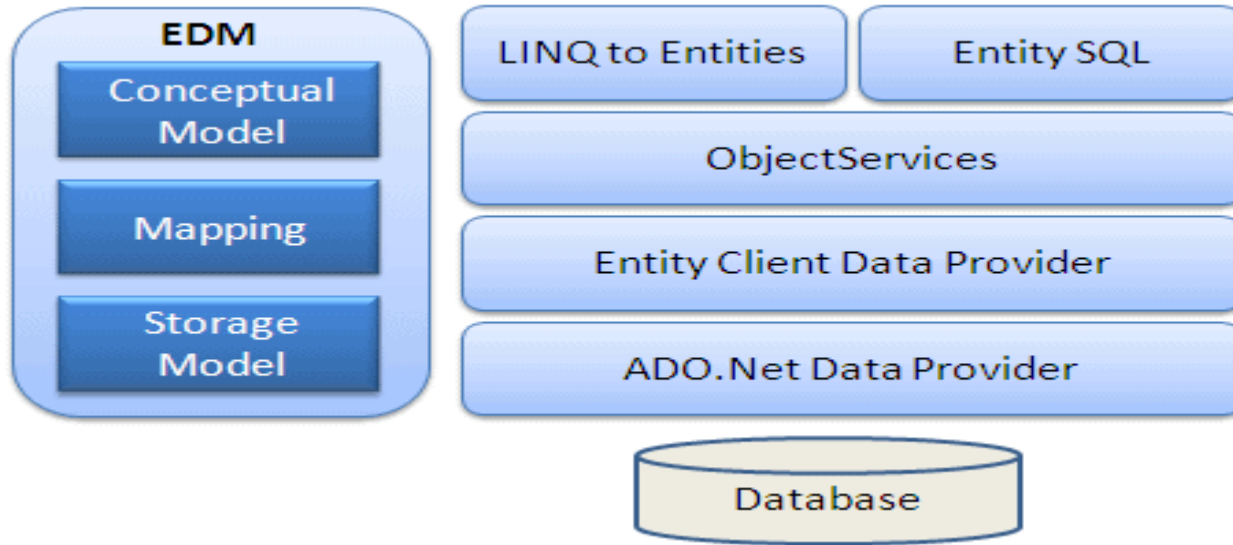
- ▶ **Cross-platform:** EF Core is a cross-platform framework which can run on Windows, Linux and Mac.
- ▶ **Modelling:** EF (Entity Framework) creates an EDM (Entity Data Model) based on POCO (Plain Old CLR Object) entities with get/set properties of different data types. It uses this model when querying or saving entity data to the underlying database.
- ▶ **Querying:** EF allows us to use LINQ queries (C#/VB.NET) to retrieve data from the underlying database. The database provider will translate this LINQ queries to the database-specific query language (e.g. SQL for a relational database). EF also allows us to execute raw SQL queries directly to the database.
- ▶ **Change Tracking:** EF keeps track of changes occurred to instances of your entities (Property values) which need to be submitted to the database.
- ▶ **Saving:** EF executes INSERT, UPDATE, and DELETE commands to the database based on the changes occurred to your entities when you call the SaveChanges() method. EF also provides the asynchronous SaveChangesAsync() method.
- ▶ **Concurrency:** EF uses Optimistic Concurrency by default to protect overwriting changes made by another user since data was fetched from the database

# EF - Features

- ▶ **Transactions:** EF performs automatic transaction management while querying or saving data. It also provides options to customize transaction management.
- ▶ **Caching:** EF includes first level of caching out of the box. So, repeated querying will return data from the cache instead of hitting the database.
- ▶ **Built-in Conventions:** EF follows conventions over the configuration programming pattern, and includes a set of default rules which automatically configure the EF model.
- ▶ **Configurations:** EF allows us to configure the EF model by using data annotation attributes or Fluent API to override default conventions.
- ▶ **Migrations:** EF provides a set of migration commands that can be executed on the NuGet Package Manager Console or the Command Line Interface to create or manage underlying database Schema.



# EF - Architecture



- ▶ **EDM (Entity Data Model):** EDM consists of three main parts - Conceptual model, Mapping and Storage model.

# EF -Architecture

- ▶ **Conceptual Model:** The conceptual model contains the model classes and their relationships. This will be independent from your database table design.
- ▶ **Storage Model:** The storage model is the database design model which includes tables, views, stored procedures, and their relationships and keys.
- ▶ **Mapping:** Mapping consists of information about how the conceptual model is mapped to the storage model.
- ▶ **LINQ to Entities:** LINQ-to-Entities (L2E) is a query language used to write queries against the object model. It returns entities, which are defined in the conceptual model. You can use your LINQ skills here.
- ▶ **Entity SQL:** Entity SQL is another query language (For EF 6 only) just like LINQ to Entities. However, it is a little more difficult than L2E and the developer will have to learn it separately.
- ▶ **Object Service:** Object service is a main entry point for accessing data from the database and returning it back. Object service is responsible for materialization, which is the process of converting data returned from an entity client data provider (next layer) to an entity object structure.

# EF -Architecture

- ▶ **Entity Client Data Provider:** The main responsibility of this layer is to convert LINQ-to-Entities or Entity SQL queries into a SQL query which is understood by the underlying database. It communicates with the ADO.Net data provider which in turn sends or retrieves data from the database.
- ▶ **ADO.Net Data Provider:** This layer communicates with the database using standard ADO.Net.

## Diff. EF Core and EF 6

| EF Core                           | EF 6                               |
|-----------------------------------|------------------------------------|
| It is cross platform.             | It is for windows only             |
| Designed for .NET Framework.      | Designed for .NET Core.            |
| Optimized for performance.        | Less Optimized.                    |
| Regular updates with new features | Limited updates.                   |
| Supports command batching.        | Does not support command batching. |

# EF Core Installation

## Package required :

- ▶ **Microsoft.EntityFrameworkCore.Tools**

The Entity Framework Core tools help with design-time development tasks. They're primarily used to manage Migrations and to scaffold a DbContext and entity types by reverse engineering the schema of a database.

- ▶ **Microsoft.EntityFrameworkCore.SqlServer**

Microsoft.EntityFrameworkCore.SqlServer is the EF Core database provider package for Microsoft SQL Server and Azure SQL.

# EF Core Installation

- ▶ **DbContext** : The DbContext class is a core component of Entity Framework Core (EF Core) that acts as a bridge between your application's domain (entities) classes and the underlying database. It manages database connections, performs CRUD (Create, Read, Update, Delete) operations, manages transactions, and tracks changes to entities. The DbContext class is responsible for interacting with the database using the configured database provider (e.g., SQL Server, SQLite, etc.).
- ▶ **DbSet**: The [DbSet<TEntity>](#) class represents an entity set that can be used for create, read, update, and delete operations.

# EF Core Installation

```
// Import the Entity Framework Core namespace to access DbContext and other EF Core functionalities.
using Microsoft.EntityFrameworkCore;

namespace EFCoreCodeFirstDemo.Entities
{
 // EFCoreDbContext class inherits from DbContext, which is the primary class for interacting with the
 // database using EF Core.
 public class EFCoreDbContext : DbContext
 {
 // Constructor that accepts DbContextOptions<EFCoreDbContext> as a parameter.
 // The options parameter contains the settings required by EF Core to configure the DbContext,
 // such as the connection string and provider.
 public EFCoreDbContext(DbContextOptions<EFCoreDbContext> options)
 : base(options) // The base(options) call passes the options to the base DbContext class
 {
 // DbSet<Student> Students represents a table in the database corresponding to the Student
 // entity.
 // EF Core uses DbSet<TEntity> to track changes and execute queries related to the Student
 // entity.
 public DbSet<Student> Students { get; set; }

 // DbSet<Branch> Branches represents a table in the database corresponding to the Branch entity.
 // Similar to the Students DbSet, this property is used by EF Core to track and manage Branch
 // entities.
 public DbSet<Branch> Branches { get; set; }

 public DbSet<Product> Products { get; set; }
 }
 }
}
```

```
builder.Services.AddDbContext<ApplicationDbContext>(options =>
{
 options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection"));
});
```

```
"ConnectionStrings": {
 "DefaultConnection":
 "Server=.;\\SQLEXPRESS;Database=ProductCrud;Trusted_Connection=True;TrustServerCertificate=True;"
}
```

# EF Core Migration

- ▶ **Create Migration:**

CMD or Bash :- **dotnet ef migrations add migration-name**

Package Manager Console :- **Add-Migration migration-name**

- ▶ **Update Database:**

CMD or Bash :- **dotnet ef database update**

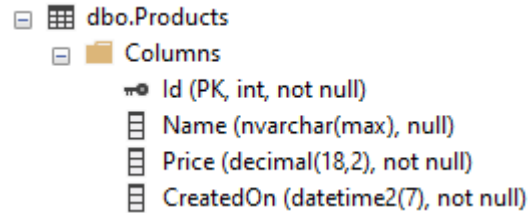
Package Manager Console :- **Update-Database**



# EF - Model DataAnnotations

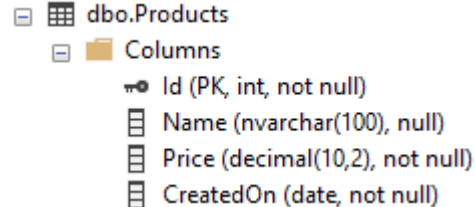
## Without DataAnnotations

```
public class Product
{
 0 references
 public int Id { get; set; }
 0 references
 public string Name { get; set; }
 0 references
 public decimal Price { get; set; }
 0 references
 public DateTime CreatedOn { get; set; }
}
```



## With DataAnnotations

```
public class Product
{
 [Key]
 0 references
 public int Id { get; set; }
 [MaxLength(100)]
 0 references
 public string Name { get; set; }
 [Column(TypeName = "decimal(10, 2)")]
 0 references
 public decimal Price { get; set; }
 [Column(TypeName = "date")]
 0 references
 public DateTime CreatedOn { get; set; }
}
```



# In this session

## What we are learning in this session

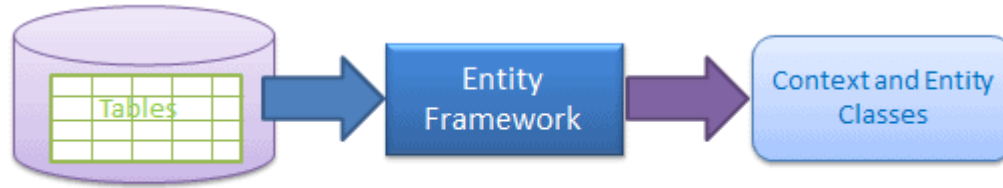
- ▶ EF Development Approach
- ▶ Introduction to EF Entities Relationships
- ▶ One to One Relationship
- ▶ One to Many Relationship
- ▶ Many to Many Relationship

# EF - Development Approach

- ▶ There are three different approaches you can use while developing your application using Entity Framework:
  - Database-First
  - Code-First
  - Model-First

# EF - Database-First Approach

- ▶ In the database-first development approach, you generate the context and entities for the existing database using EDM wizard integrated in Visual Studio or executing EF commands.

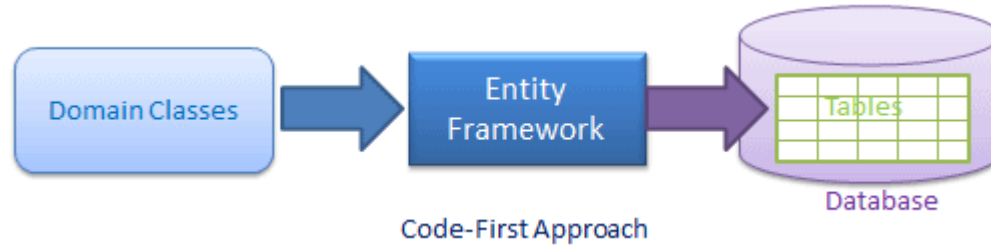


Database-First Approach

- ▶ EF 6 supports the database-first approach extensively. Visit EF 6 DB-First section to learn about the database-first approach using EF 6.
- ▶ EF Core includes limited support for this approach.

# EF - Code-First Approach

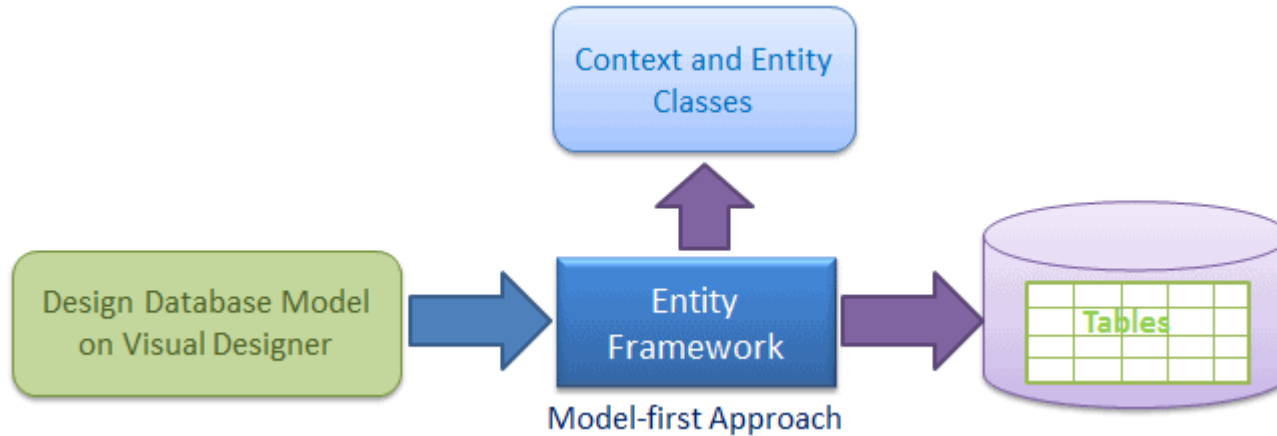
- ▶ Use this approach when you do not have an existing database for your application. In the code-first approach, you start writing your entities (domain classes) and context class first and then create the database from these classes using migration commands.



- ▶ Developers who follow the Domain-Driven Design (DDD) principles, prefer to begin with coding their domain classes first and then generate the database required to persist their data.

# EF - Model-First Approach

- ▶ In the model-first approach, you create entities, relationships, and inheritance hierarchies directly on the visual designer integrated in Visual Studio and then generate entities, the context class, and the database script from your visual model.



- ▶ EF 6 includes limited support for this approach.
- ▶ EF Core does not support this approach.

# Relationships between Entities

- ▶ In relational databases, relationships between tables (or entities) define how data in one table relates to data in another table.
- ▶ Relationships or associations between entities are defined in a database by the existence of Foreign Keys.
- ▶ When we create relationships between two entities in Entity Framework Core, one entity serves as the **Principal Entity**, and the other entity serves as the **Dependent Entity**.

# Relationships between Entities

- ▶ **Navigation Properties:** Navigation Properties in entity classes allow users to navigate from one entity to a related entity or collection of entities.

**Reference Navigation Property:** It represents a single related entity, typically in a one-to-one relationship. For example, a Customer might have exactly one Address.

**Collection Navigation Property:** It represents a collection of related entities commonly used in one-to-many or many-to-many relationships. For example, a Customer might have many Orders.



# Types of Relationships

- ▶ **One-to-One (1:1):** A one-to-one (or more usually a one-to-zero or one) relationship exists when only one row of data in the principal table is linked to zero or one row in a dependent table.
- ▶ **One-to-Many:** In a one-to-many relationship, one entity instance (the principal) relates to multiple instances of another entity (the dependents). Typically, the principal entity holds the primary key, while the dependent entity holds the foreign key.
- ▶ **Many-to-Many:** In this relationship, both entities have collection navigation properties, and EF Core creates an implicit join table in the database to link the two.

# One to One Relationships

```
public class Student
{
 [Key]
 public int StudentId { get; set; }
 public string Name { get; set; }

 public StudentAddress Address { get; set; }
}
```

```
public class StudentAddress
{
 public int Id { get; set; }
 public string Street { get; set; }
 public string City { get; set; }

 // Foreign Key
 [ForeignKey("StudentId")]
 public int StudentId { get; set; }

 // Navigation Property
 public Student Student { get; set; }
}
```

# One to Many Relationships

```
public class Course
{
 public int Id { get; set; }
 public string Name { get; set; }

 // Navigation property (One Course can have many Students)
 public List<Student> Students { get; set; } = new List<Student>();
}
```

```
public class Student
{
 [Key]
 public int StudentId { get; set; }
 public string Name { get; set; }

 public StudentAddress Address { get; set; }

 [ForeignKey("CourseId")]
 public int? CourseId { get; set; }

 public Course course { get; set; }
}
```

# Many to Many Relationships

```
public class Student
{
 public int Id { get; set; }
 public string Name { get; set; }

 // Many-to-Many relationship with Clubs
 public List<Club> Clubs { get; set; } = new List<Club>();
}
```

```
public class Club
{
 public int Id { get; set; }
 public string Name { get; set; }

 // Many-to-Many relationship with Students
 public List<Student> Students { get; set; } = new List<Student>();
}
```

# Entity States

- ▶ In Entity Framework Core (EF Core), entities can exist in various states during their lifecycle as the DbContext tracks them.
- ▶ Entity States in EF Core represent an entity's state in relation to its interaction with the DbContext. EF Core uses these states to determine what operations need to be performed on the database when SaveChanges() is called.
- ▶ **Added:** The entity is being tracked by the context but does not yet exist in the database. When SaveChanges() is executed, EF Core issues an INSERT command for this entity.
- ▶ **Unchanged:** The entity is being tracked and exists in the database but has not been modified since it was retrieved. EF Core will not issue any database commands for entities in this state during SaveChanges().

# Entity States

- ▶ **Modified:** The entity is being tracked and exists in the database, and some or all of its property values have been modified. When `SaveChanges()` is executed, EF Core issues a `UPDATE` command that only updates the changed properties.
- ▶ **Deleted:** The entity is being tracked and exists in the database but has been marked for deletion. When `SaveChanges()` is executed, EF Core issues a `DELETE` command for this entity.
- ▶ **Detached:** The entity is not being tracked by the context. EF Core will not perform any database operations on detached entities when `SaveChanges()` is called. This state is typical for entities that have been created but not yet added to the context, have been explicitly detached from the context, or have been deleted from the database.
- ▶ We can check the current state of my entity using `_context.Entry(trainer).State;`

# Eager Loading

- ▶ In Eager Loading, related data (such as Branch, Address, and Courses) is loaded automatically as part of the initial query to the database. This is done using the `Include()` and `ThenInclude()` methods. The related entities are loaded in the same query when the Student entity is retrieved.
- ▶ This helps avoid the “**N+1 Query Problem**,” where multiple queries would otherwise be sent to the database to fetch each related entity one by one.
- ▶ Eager Loading is ideal when you know we need the related entities along with the main entity immediately and want to minimize the number of database queries.
- ▶ **When to Use Eager Loading with EF Core:**
  - You Know You'll Need Related Data Immediately:
  - You Want to Minimize Round Trips to the Database:
  - When You Want to Avoid the N+1 Query Problem:

# Lazy Loading

- ▶ In Lazy Loading, the related data is not retrieved initially. Instead, the related entities are loaded only when we access the navigation property for the first time. EF Core makes a separate query to the database when we try to access the related property.
- ▶ **How Do We Implement Lazy Loading in EF Core?**
  - Install the Required Package** - `Microsoft.EntityFrameworkCore.Proxies`
  - Enable Lazy Loading in the DbContext:** `optionsBuilder.UseLazyLoadingProxies();`
  - Mark Navigation Properties as virtual**
- ▶ With Lazy Loading enabled in EF Core, EF Core will automatically load the data from the database when we access a related entity for the first time. The loaded data is then stored in the context's change tracker (in memory). If we reaccess the same related entity within the same DbContext instance, the data is fetched from in-memory rather than querying the database again. This avoids redundant database calls and improves performance for subsequent access to the same entity.



# Explicit Loading

- ▶ Explicit Loading in Entity Framework Core is a technique where related entities are loaded manually after the main entity has already been retrieved from the database.
- ▶ This allows us to defer loading related entities until they are needed.
- ▶ **How we can Implement Explicit Loading:**  
Explicit loading involves methods like `Entry()`, `Reference()`, `Collection()`, and `Load()`.

```
context.Entry(student); // Get the entry for the student entity

context.Entry(student).Reference(s => s.Branch).Load(); // Load a single related entity (Branch)

context.Entry(student).Collection(s => s.Courses).Load(); // Load a collection of related entities (Courses)

context.Entry(student).Reference(s => s.Branch).Load(); // Loads the related Branch entity
```

# Design Patterns

- **Definition**

Design patterns are reusable solutions to common software design problems.

- **Purpose:**

Improve code reusability, scalability, and maintainability.

- **Categories:**

Creational, Structural, and Behavioral Patterns.

# Types of Design Pattern

- **Creational Design Patterns**

Creational design patterns provide various object creation mechanisms, which increase flexibility and reuse of existing code (e.g., Singleton, Factory Method, Abstract Factory).

- **Structural Design Patterns**

Structural design patterns explain how to assemble objects and classes into larger structures, while keeping these structures flexible and efficient. e.g., Adapter, Composite, Decorator)

- **Behavioral Design Patterns:**

Behavioral design patterns are concerned with algorithms and the assignment of responsibilities between objects (e.g., Observer, Command, Iterator).

# Singleton

Singleton design pattern in C# is one of the most popular design patterns. In this pattern, a class has only one instance in the program that provides a global point of access to it. In other words, a singleton is a class that allows only a single instance of itself to be created and usually gives simple access to that instance.

There are various ways to implement a singleton pattern in C#. The following are the common characteristics of a singleton pattern.

- Private and parameterless single constructor
- Sealed class.
- Static variable to hold a reference to the single created instance
- A public and static way of getting the reference to the created instance.

# Singleton

## Advantages of Singleton Design Pattern

The advantages of a Singleton Pattern are,

- Singleton pattern can implement interfaces.
- Can be lazy-loaded and has Static Initialization.
- It helps to hide dependencies.
- It provides a single point of access to a particular instance, so it is easy to maintain.

## Disadvantages of Singleton Design Pattern

The disadvantages of a Singleton Pattern are,

- Unit testing is a bit difficult as it introduces a global state into an application
- Reduces the potential for parallelism within a program by locking.

# Singleton

## How to Implement Singleton Pattern in C# code

There are several ways to implement a Singleton Pattern in C#.

- No Thread Safe Singleton.
- Thread-Safety Singleton.
- Thread-Safety Singleton using Double-Check Locking.
- Thread-safe without a lock.
- Using .NET 4's Lazy<T> type.

# Singleton

## No Thread Safe Singleton

Explanation of the following code,

- The following code is not thread-safe.
- Two different threads could both have evaluated the test (if instance == null) and found it to be true, then both create instances, which violates the singleton pattern.

```
public sealed class Singleton1 {
 private Singleton1() {}
 private static Singleton1 instance = null;
 public static Singleton1 Instance {
 get {
 if (instance == null) {
 instance = new Singleton1();
 }
 return instance;
 }
 }
}
```

# Singleton

## Thread Safety Singleton

### Explanation of the following code

- The following code is thread-safe.
- In the code, the thread is locked on a shared object and checks whether an instance has been created or not. It takes care of the memory barrier issue and ensures that only one thread will create an instance. For example: Since only one thread can be in that part of the code at a time, by the time the second thread enters it, the first thread will have created the instance, so the expression will evaluate as false.
- The biggest problem with this is performance; performance suffers since a lock is required every time an instance is requested.

```
public sealed class Singleton2 {
 Singleton2() {}
 private static readonly object lock = new object();
 private static Singleton2 instance = null;
 public static Singleton2 Instance {
 get {
 lock(lock) {
 if (instance == null) {
 instance = new Singleton2();
 }
 return instance;
 }
 }
 }
}
```



# Factory Design Pattern

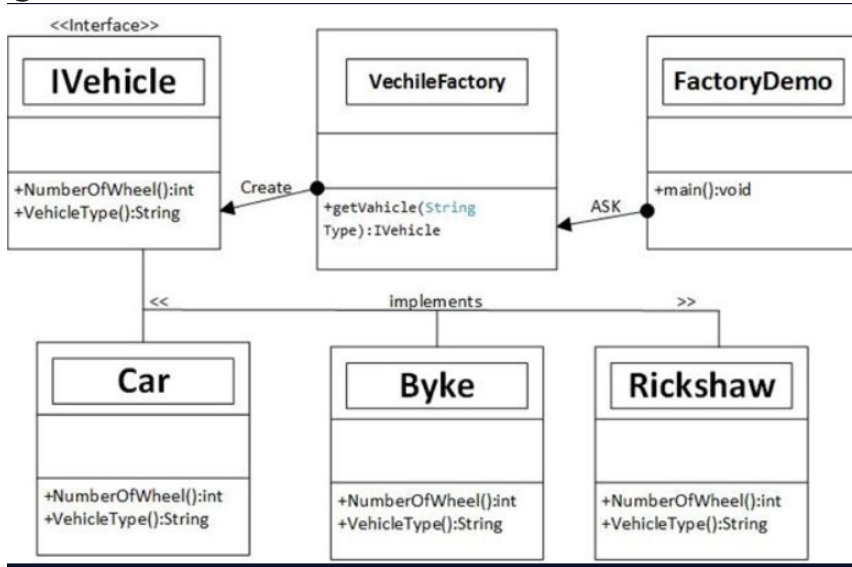
This pattern comes under the creational patterns which are widely used in programming constructs. In other words, “This Pattern provides the best way to create the Objects. In Factory patterns we create the object of the Class without Exposing the Creation/Instantiation Logic to the User who wants to create the Object and then return the newly Created object using the Common Interface which is inherited by the Class”.

There are various ways to implement a singleton pattern in C#. The following are the common characteristics of a singleton pattern.

- Private and parameterless single constructor
- Sealed class.
- Static variable to hold a reference to the single created instance
- A public and static way of getting the reference to the created instance.

# Factory Design Pattern

We take the example of a vehicle. As we know, this vehicle is the superior class of all kinds of the vehicles and they have some common attributes such as engine, wheels, gears etc.



# Factory Design Pattern

- First of all, we create the Interface named as “IVehicle” which contains two member -- the first one is NumberOfWheels() which Returns the Number of wheels specific to the Vehicle Type the second one is VehicleType() which describes the Type of the Vehicle either (Car/Bike/Rickshaw).
- In our Project the First Class is the Program.cs which calls the object of the VehicleFactory.
- First of all they Get the Input From the Console whether user wants either car/Bike/Rickshaw, then the information is passed to the VehicleFactory class method name as getVehicle which returns the specific object of the car/Bike/Rickshaw according to the condition match but we see that the all classes inherit the IVehicle interface and according to Factory pattern “a Newly created Object using the Common interface is returned” so object is returned in the form of IVEchile interface.

# Factory Design Pattern

IVehicle.cs

Car.cs

```
using System;
namespace FactoryPattern
{
 6 references
 public interface IVehicle
 {
 4 references
 String VehicleType();
 4 references
 int NumberOfWheels();
 }
}
```

Bike.cs

```
using System;
namespace FactoryPattern
{
 2 references
 class Bike:IVehicle
 {
 private readonly int Wheels;
 1 reference
 public Bike()
 {
 this.Wheels = 2;
 }
 4 references
 public int NumberOfWheels()
 {
 return this.Wheels;
 }
 4 references
 public string VehicleType()
 {
 return "Byke class";
 }
 }
}
```

```
using System;
namespace FactoryPattern
{
 2 references
 class Car : IVehicle
 {
 private readonly int Wheels;
 1 reference
 public Car()
 {
 this.Wheels = 4;
 }
 4 references
 public int NumberOfWheels()
 {
 return this.Wheels;
 }
 4 references
 public string VehicleType()
 {
 return "Car class";
 }
 }
}
```

Same way we create a Rickshaw.cs

# Factory Design Pattern

VehicleFactory.cs

```
using System;
namespace FactoryPattern
{
 1 reference
 class VehicleFactory
 {
 1 reference
 public static IVehicle getVahicle(String Type)
 {
 IVehicle ObjectType = null;
 if (Type.ToLower().Equals("bike"))
 {
 ObjectType= new Bike();
 }
 else if (Type.ToLower().Equals("car"))
 {
 ObjectType = new Car();
 }
 else if (Type.ToLower().Equals("rikshaw"))
 {
 ObjectType = new Rickshaw();
 }
 return ObjectType;
 }
 }
}
```

# Factory Design Pattern

Use the Factory class to create the Object

```
using System;
namespace FactoryPattern
{
 O references
 class Program
 {
 O references
 static void Main(string[] args)
 {
 String inputFromConsole = null;
 Console.WriteLine("Enter the Vahicle Type");
 inputFromConsole= Console.ReadLine();
 IVehicle Type = VehicleFactory.getVahicle(inputFromConsole);
 Console.WriteLine("Type is "+Type.VehicleType());
 Console.WriteLine("Number of Wheel is " + Type.NumberOfWheels());
 Console.ReadKey();
 }
 }
}
```

# CQRS Pattern

**The Command and Query Responsibility Segregation (CQRS) pattern states that we must separate the operations for reading the data from the operations for writing or updating the data. This means that functions for reading and writing data are not kept in the same interface or class. The main advantages of doing this include:**

- **Separate teams can work on these operations**
- **Each can be made to scale according to their own needs. Write operations are mostly used much less than read operations**
- **Each can have their own security as per requirements**
- **Read operations can have a different architecture to support caching, conversions to data transformation objects as required by clients**
- **Write operations can include data validation. lookups etc.**

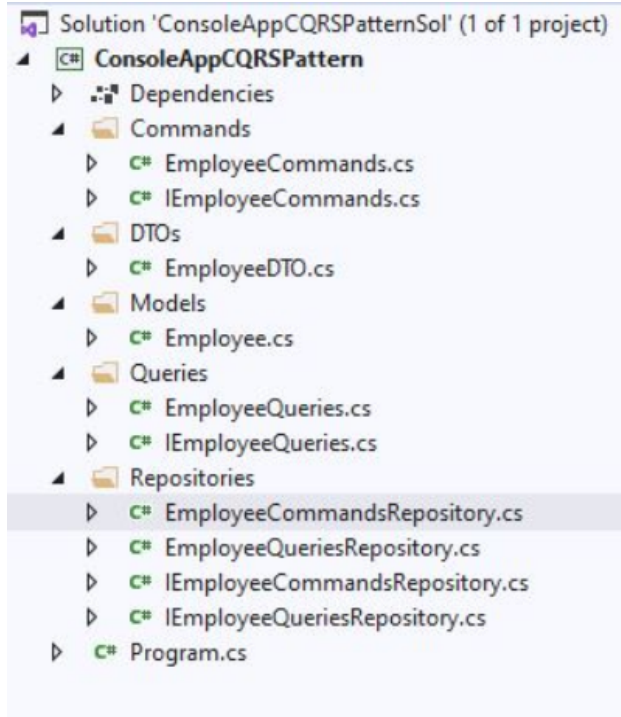
## CQRS Pattern

However, do keep in mind that this pattern is better suited to larger applications where the requirements and load levels between read and write operations are different. For a simple and small application, the normal CRUD pattern, often auto-generated from ORM tools, is sufficient.



# CQRS Pattern

The structure of the completed solution looks like the shown below:



# CQRS Pattern

## The Commands (Write) repository

```
using ConsoleAppCQRSPattern.Models;
namespace ConsoleAppCQRSPattern.Repositories {
 public interface IEmployeeCommandsRepository {
 void SaveEmployee(Employee employee);
 }
}

using ConsoleApp CQRSPattern.Models;
namespace ConsoleApp CQRSPattern.Repositories {
 public class EmployeeCommandsRepository: IEmployeeCommandsRepository {
 public void SaveEmployee(Employee employee) {
 //Persist the employee record in a data store
 }
 }
}
```

# CQRS Pattern

## The Queries (Read) repository

```
using ConsoleAppCQRSPattern.Models;
namespace ConsoleAppCQRSPattern.Repositories {
 public interface IEmployeeQueriesRepository {
 Employee GetById(int employeeID);
 }
}

using System;
using ConsoleAppCQRSPattern.Models;
namespace ConsoleAppCQRSPattern.Repositories {
 public class EmployeeQueriesRepository: IEmployeeQueriesRepository {
 public Employee GetById(int employeeID) {
 // Get the employee record from a data store
 // Below is for demo purposes only
 return new Employee() {
 Id = 100,
 FirstName = "John",
 LastName = "Smith",
 DateOfBirth = new DateTime(2000, 1, 1),
 Street = "100 Toronto Street",
 City = "Toronto",
 PostalCode = "k1k 1k1"
 };
 }
 }
}
```

# CQRS Pattern

## The Queries (Read) repository

```
using ConsoleAppCQRSPattern.Models;
namespace ConsoleAppCQRSPattern.Repositories {
 public interface IEmployeeQueriesRepository {
 Employee GetById(int employeeID);
 }
}

using System;
using ConsoleAppCQRSPattern.Models;
namespace ConsoleAppCQRSPattern.Repositories {
 public class EmployeeQueriesRepository: IEmployeeQueriesRepository {
 public Employee GetById(int employeeID) {
 // Get the employee record from a data store
 // Below is for demo purposes only
 return new Employee() {
 Id = 100,
 FirstName = "John",
 LastName = "Smith",
 DateOfBirth = new DateTime(2000, 1, 1),
 Street = "100 Toronto Street",
 City = "Toronto",
 PostalCode = "k1k 1k1"
 };
 }
 }
}
```

# CQRS Pattern

Next, we look at the two middle-tier components (These are in the Queries and Commands folders),

**Queries (To read data)**

```
using ConsoleAppCQRSPattern.DTOs;
namespace ConsoleAppCQRSPattern.Queries {
 public interface IEmployeeQueries {
 EmployeeDTO FindByID(int employeeID);
 }
}

using System;
using ConsoleAppCQRSPattern.Repositories;
using ConsoleAppCQRSPattern.DTOs;
namespace ConsoleAppCQRSPattern.Queries {
 public class EmployeeQueries {
 private readonly IEmployeeQueriesRepository _repository;
 public EmployeeQueries(IEmployeeQueriesRepository repository) {
 _repository = repository;
 }

 public EmployeeDTO FindByID(int employeeID) {
 var emp = _repository.GetByID(employeeID);
 return new EmployeeDTO {
 Id = emp.Id,
 FullName = emp.FirstName + " " + emp.LastName,
 Address = emp.Street + " " + emp.City + " " + emp.PostalCode,
 Age = Convert.ToInt32(Math.Abs(((DateTime.Now - emp.DateOfBirth).TotalDays) /
 });
 }
 }
}
```

# CQRS Pattern

## Commands (To write data)

```
using ConsoleAppCQRSPattern.Models;
namespace ConsoleAppCQRSPattern.Commands {
 public interface IEmployeeCommands {
 void SaveEmployeeData(Employee employee);
 }
}

using ConsoleAppCQRSPattern.Models;
using ConsoleAppCQRSPattern.Repositories;
namespace ConsoleAppCQRSPattern.Commands {
 public class EmployeeCommands: IEmployeeCommands {
 private readonly IEmployeeCommandsRepository _repository;
 public EmployeeCommands(IEmployeeCommandsRepository repository) {
 _repository = repository;
 }
 public void SaveEmployeeData(Employee employee) {
 _repository.SaveEmployee(employee);
 }
 }
}
```

# CQRS Pattern

```
namespace ConsoleAppCQRSPattern.DTOs {
 public class EmployeeDTO {
 public int Id {
 get;
 set;
 }
 public string FullName {
 get;
 set;
 }
 public int Age {
 get;
 set;
 }
 public string Address {
 get;
 set;
 }
 }
}
```

```
using System;
namespace ConsoleAppCQRSPattern.Models {
 public class Employee {
 public int Id {
 get;
 set;
 }
 public string FirstName {
 get;
 set;
 }
 public string LastName {
 get;
 set;
 }
 public DateTime DateOfBirth {
 get;
 set;
 }
 public string Street {
 get;
 set;
 }
 public string City {
 get;
 set;
 }
 public string PostalCode {
 get;
 set;
 }
 }
}
```

# CQRS Pattern

Finally, we call these operations from the main Program class. Please note that in this demo application, we create an instance of the repository directly. In a real-world scenario, we would probably use some dependency injection framework to do this.

```
1 using ConsoleAppCQRSPattern.Commands;
2 using ConsoleAppCQRSPattern.Queries;
3 using ConsoleAppCQRSPattern.Repositories;
4 using System;
5 namespace ConsoleAppCQRSPattern {
6 class Program {
7 static void Main(string[] args) {
8 // Command the Employee Domain to save data
9 var employeeCommand = new EmployeeCommands(new EmployeeCommandsRepository());
10 employeeCommand.SaveEmployeeData(new Models.Employee {
11 Id = 200, FirstName = "Jane", LastName = "Smith", Street = "150 Toronto Street",
12 });
13 Console.WriteLine($"Employee data stored");
14 // Query the Employee Domain to get data
15 var employeeQuery = new EmployeeQueries(new EmployeeQueriesRepository());
16 var employee = employeeQuery.FindById(100);
17 Console.WriteLine($"Employee ID:{employee.Id}, Name:{employee.FullName}, Address:{emp
18 Console.ReadKey();
19 }
20 }
21 }
```